

REFERENCE

Hooks reference

 Copy page

Reference for Claude Code hook events, configuration schema, JSON input/output formats, exit codes, async hooks, HTTP hooks, prompt hooks, and MCP tool hooks.

💡 For a quickstart guide with examples, see [Automate actions with hooks](#).

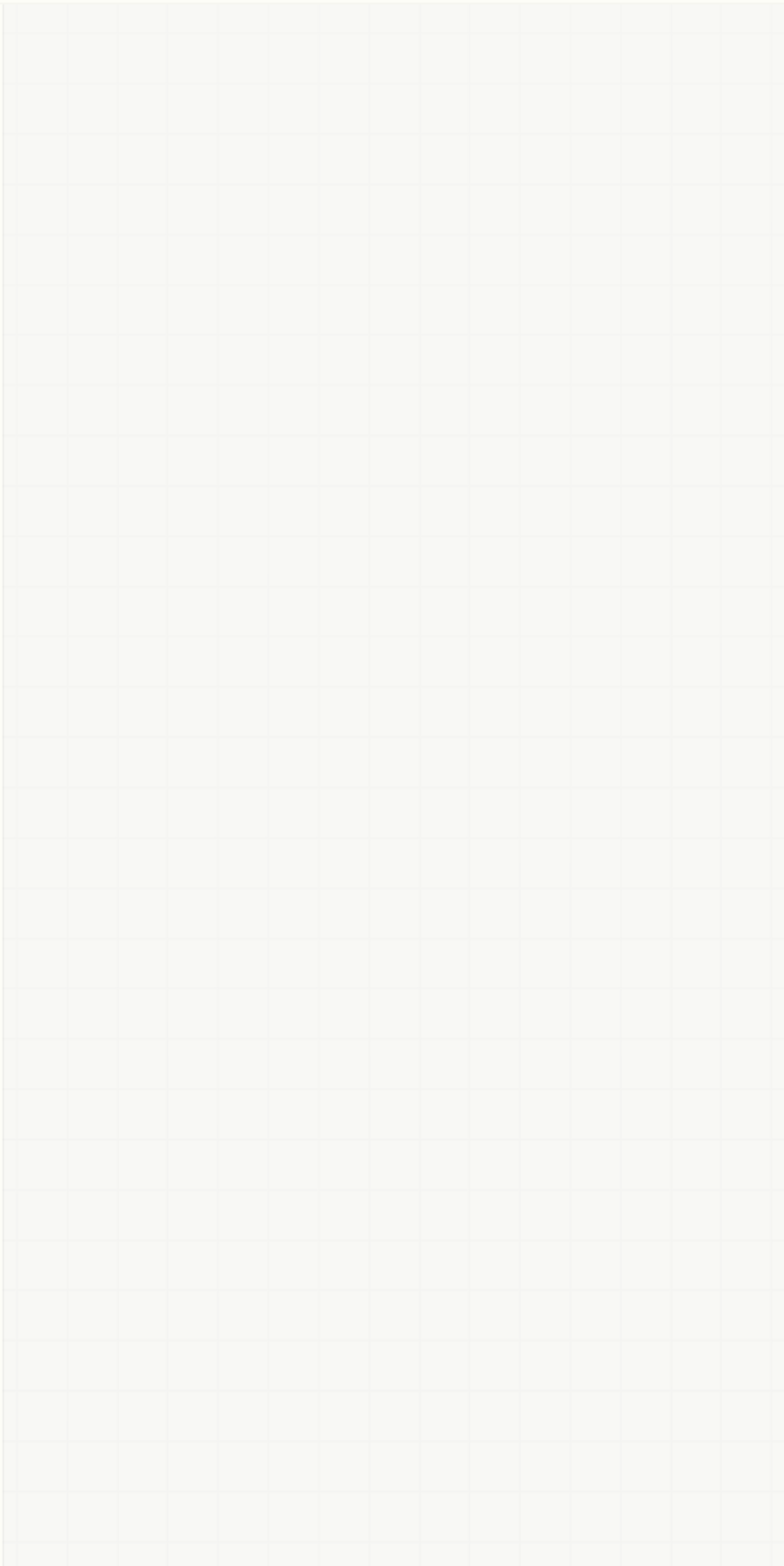
Hooks are user-defined shell commands, HTTP endpoints, or LLM prompts that execute automatically at specific points in Claude Code's lifecycle. Hooks run wherever Claude Code runs: sessions in the terminal, IDE extensions, the [Desktop app](#), and [Claude Code on the web](#) all fire the same hook events. Use this reference to look up event schemas, configuration options, JSON input/output formats, and advanced features like async hooks, HTTP hooks, and MCP tool hooks.

Hook lifecycle

Claude Code runs hooks at specific points during a session. When an event fires and a matcher matches, Claude Code passes JSON context about the event to your hook handler. For command hooks, input arrives on stdin. For HTTP hooks, it arrives as the POST request body. Your handler can then inspect the input, take action, and optionally return a decision.

Events fall into three cadences:

- once per session: `SessionStart` and `SessionEnd`
- once per turn: `UserPromptSubmit` , `Stop` , and `StopFailure`
- on every tool call inside the agentic loop: `PreToolUse` and `PostToolUse` , except `EndConversation` calls, which skip both



Event	When it fires
SessionStart	When a session begins or resumes
Setup	When you start Claude Code with <code>--init-only</code> , or with <code>--init</code> or <code>--maintenance</code> in <code>-p</code> mode. For one-time preparation in CI or scripts
UserPromptSubmit	When you submit a prompt, before Claude processes it
UserPromptExpansion	When a user-typed command expands into a prompt, before it reaches Claude. Can block the expansion
PreToolUse	Before a tool call executes. Can block it
PermissionRequest	When a tool call needs a permission decision
PermissionDenied	When auto mode denies a tool call, including denials without a classifier verdict. Use JSON <code>hookSpecificOutput.retry: true</code> to tell the model it may retry the denied tool call. Claude Code ignores <code>retry</code> when the classifier produced no verdict
PostToolUse	After a tool call succeeds
PostToolUseFailure	After a tool call fails
PostToolBatch	After a full batch of parallel tool calls resolves, before the next model call
Notification	When Claude Code sends a notification
MessageDisplay	While assistant message text is displayed
SubagentStart	When a subagent is spawned
SubagentStop	When a subagent finishes
TaskCreated	When a task is being created via <code>TaskCreate</code>
TaskCompleted	When a task is being marked as completed
Stop	When Claude finishes responding
StopFailure	When the turn ends due to an API error
TeammateIdle	When an <u>agent team</u> teammate is about to go idle
InstructionsLoaded	When a <code>CLAUDE.md</code> or <code>.claude/rules/*.md</code> file is loaded into context. Fires at session start and when files are lazily loaded during a session
ConfigChange	When a configuration file changes during a session

`register_repo_root` control request

<code>FileChanged</code>	When a watched file changes on disk. The <code>matcher</code> field specifies which filenames to watch
<code>WorktreeCreate</code>	When a worktree is being created via <code>--worktree</code> , <code>isolation: "worktree"</code> , or for a background session. Replaces default git behavior
<code>WorktreeRemove</code>	When a worktree is being removed at session exit, when a subagent finishes, or when you delete a background session
<code>PreCompact</code>	Before context compaction
<code>PostCompact</code>	After context compaction completes
<code>Elicitation</code>	When an MCP server requests user input during a tool call
<code>ElicitationResult</code>	After a user responds to an MCP elicitation, before the response is sent back to the server
<code>SessionEnd</code>	When a session terminates

How a hook resolves

To see how the event, the matcher, and the handler fit together, consider this `PreToolUse` hook that blocks destructive shell commands.

macOS/Linux **Windows (PowerShell)**

The `matcher` narrows to Bash tool calls and the `if` condition narrows further to Bash subcommands matching `rm *`, so `block-rm.sh` only spawns when both filters match:

```
"matcher": "Bash",
"hooks": [
  {
    "type": "command",
    "if": "Bash(rm *)",
    "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/block-rm.sh",
    "args": []
  }
]
}
```

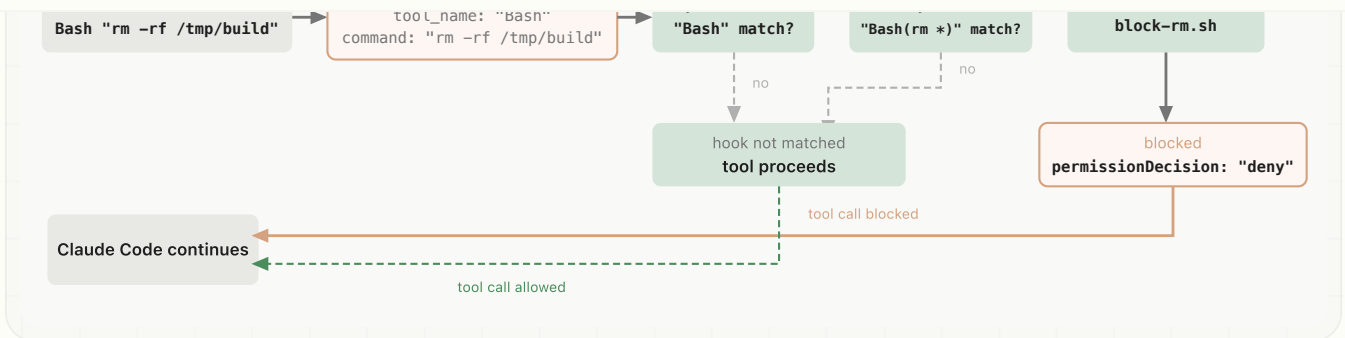
The script reads the JSON input from stdin, extracts the command, and returns a

permissionDecision of "deny" if it contains `rm -rf`. Save it to `.claude/hooks/block-rm.sh` in your project and make it executable with `chmod +x .claude/hooks/block-rm.sh` so Claude Code can run it:

```
#!/bin/bash
# .claude/hooks/block-rm.sh
COMMAND=$(jq -r '.tool_input.command')

if echo "$COMMAND" | grep -q 'rm -rf'; then
  jq -n '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "deny",
      permissionDecisionReason: "Destructive command blocked by hook"
    }
  }'
else
  exit 0 # no decision; normal permission flow applies
fi
```

This script, like the other Bash examples on this page that parse JSON input, uses `jq`, so install `jq` and make sure it is on your `PATH` before trying them.



1 Event fires

The `PreToolUse` event fires. Claude Code sends the tool input as JSON on stdin to the hook:

```
{ "tool_name": "Bash", "tool_input": { "command": "rm -rf /tmp/build" }, ... }
```

2 Matcher checks

The matcher `"Bash"` matches the tool name, so this hook group activates. If you omit the matcher or use `"*"`, the group activates on every occurrence of the event.

3 If condition checks

The `if` condition `"Bash(rm *)"` matches because `rm -rf /tmp/build` is a subcommand matching `rm *`, so this handler spawns. If the command had been `npm test`, the `if` check would fail and `block-rm.sh` would never run, avoiding the process spawn overhead. The `if` field is optional; without it, every handler in the matched group runs.

4 Hook handler runs

The script inspects the full command and finds `rm -rf`, so it prints a decision to stdout:

```
"permissionDecisionReason": "Destructive command blocked by hook"
  }
}
```

If the command had been a safer `rm` variant like `rm file.txt`, the script would hit `exit 0` instead. Exit code 0 with no output means the hook has no decision to report, so the tool call continues through the normal permission flow. The hook can deny the call, but staying silent doesn't approve it.

5 Claude Code acts on the result

Claude Code reads the JSON decision, blocks the tool call, and shows Claude the reason.

The Configuration section below documents the full schema, and each hook event section documents what input your command receives and what output it can return.

Configuration

Hooks are defined in JSON settings files. The configuration has three levels of nesting:

1. Choose a hook event to respond to, like `PreToolUse` or `Stop`
2. Add a matcher group to filter when it fires, like "only for the Bash tool"
3. Define one or more hook handlers to run when matched

See How a hook resolves above for a complete walkthrough with an annotated example.

❗ This page uses specific terms for each level: **hook event** for the lifecycle point, **matcher group** for the filter, and **hook handler** for the shell command, HTTP endpoint, MCP tool, prompt, or agent that runs. "Hook" on its own refers to the general feature.

Hook locations

Where you define a hook determines its scope:

<code>.claude/settings.json</code>	Single project	Yes, can be committed to the repo
<code>.claude/settings.local.json</code>	Single project	No, gitignored when Claude Code saves a setting to it
Managed policy settings	Organization-wide	Yes, admin-controlled
Plugin <code>hooks/hooks.json</code>	When plugin is enabled	Yes, bundled with the plugin
Skill frontmatter	The rest of the session once the skill is invoked. See Hooks in skills and agents	Yes, defined in the skill file
Subagent frontmatter	While that subagent is running	Yes, defined in the subagent file

Cloud sessions on [Claude Code on the web](#) don't read your local `~/ .claude/settings.json` ; hooks there come from the repo and from your organization's server-managed settings. In a [self-hosted environment](#), Claude Code also runs the hooks the operator seeded from the runner host's `~/ .claude/` , and it runs the hooks in the runner image's managed settings file when that file is among the [managed sources Claude Code applies](#), which by default means only when neither server-managed settings nor an MDM-delivered Claude Code policy supplies the managed tier. See [what carries over from your setup](#) for which files reach a cloud session.

For details on settings file resolution, see [settings](#).

Hooks from settings files, managed policy settings, and plugins also run inside [subagents](#). When a subagent calls a tool, tool events such as `PreToolUse` and `PostToolUse` fire the same configured hooks as in the main conversation, and the input carries the `agent_id` and `agent_type` [common input fields](#) that identify the subagent.

Enterprise administrators can use `allowManagedHooksOnly` to restrict which hooks run:

- Your user, project, local, and plugin hooks are blocked. Hooks from plugins force-enabled in managed settings `enabledPlugins` are exempt
- Claude Code also narrows your `statusLine` , `fileSuggestion` , and `subagentStatusLine` settings to managed settings
- Claude Code also disables plugins with a `command source`, including plugins force-enabled in managed settings `enabledPlugins` , unless `disableCommandPluginSources` is explicitly set to `false` . `command` sources require Claude Code v2.1.229 or later

See `whatHooksUnder allowManagedHooksOnly`.

Hook entries merge across settings levels rather than replacing each other: user, project, and local settings add their own hooks without removing managed ones, and the `disableAllHooks` setting can't disable managed hooks from outside managed settings.

The **HTTP hook allowlists** apply to hooks from every source, including managed policy settings:

- `allowedHttpHookUrls` : when defined at any settings level, Claude Code runs an HTTP hook handler only if its URL matches the merged allowlist
- `httpHookAllowedEnvVars` : when defined, Claude Code interpolates only the environment variables on that list into hook headers

Matcher patterns

The `matcher` field filters when hooks fire. How a matcher is evaluated depends on the characters it contains:

Matcher value	Evaluated as	Example
<code>"*"</code> , <code>""</code> , or omitted	Match all	fires on every occurrence of the event
Only letters, digits, <code>_</code> , <code>-</code> , spaces, <code>,</code> , and <code> </code>	Exact string, or list of exact strings separated by <code> </code> or <code>,</code> with optional surrounding whitespace	<code>Bash</code> matches only the Bash tool; <code>Edit Write</code> and <code>Edit, Write</code> each match either tool exactly; <code>code-reviewer</code> matches only that agent type
Contains any other character	JavaScript regular expression, unanchored	<code>^Notebook</code> matches any tool whose name starts with <code>Notebook</code> ; <code>mcp__memory__.*</code> matches every tool from the <code>memory</code> server

A matcher on the regular-expression path is tested with JavaScript's `RegExp.prototype.test`, which succeeds on a match anywhere in the value. `Edit.*` matches both `Edit` and `NotebookEdit` ; wrap the pattern in `^` and `$`, as in `^Edit$`, when you need a whole-string match.

Comma separators and the surrounding whitespace tolerance require Claude Code v2.1.191 or later.

Hyphens in the exact-match set require Claude Code v2.1.195 or later. On earlier versions a hyphenated name like `code-reviewer` is evaluated as an unanchored regular expression, so it also

only. A hyphen, space, or comma in a matcher for these two events keeps it on the regular expression path, and only `|` separates alternatives. Every other event with matcher support in the table that follows accepts `|` or `,`.

The `FileChanged` event doesn't follow these rules when building its watch list. See [FileChanged](#).

Each event type matches on a different field:

Event	What the matcher filters	Example matcher values
<code>PreToolUse</code> , <code>PostToolUse</code> , <code>PostToolUseFailure</code> , <code>PermissionRequest</code> , <code>PermissionDenied</code>	tool name	<code>Bash</code> , <code>Edit Write</code> , <code>mcp__.*</code>
<code>SessionStart</code>	how the session started	<code>startup</code> , <code>resume</code> , <code>clear</code> , <code>compact</code> , <code>fork</code>
<code>Setup</code>	which CLI flag triggered setup	<code>init</code> , <code>maintenance</code>
<code>SessionEnd</code>	why the session ended	<code>clear</code> , <code>resume</code> , <code>logout</code> , <code>prompt_input_exit</code> , <code>other</code>
<code>Notification</code>	notification type	<code>permission_prompt</code> , <code>idle_prompt</code> , <code>auth_success</code> , <code>elicitation_dialog</code> , <code>elicitation_url_dialog</code> , <code>elicitation_complete</code> , <code>elicitation_response</code> , <code>agent_needs_input</code> , <code>agent_completed</code> , <code>quota_auto_resume_fired</code> , <code>quota_auto_resume_stale</code> , <code>quota_auto_resume_disabled</code>
<code>SubagentStart</code>	agent type	<code>general-purpose</code> , <code>Explore</code> , <code>Plan</code> , custom agent names, or plugin-scoped names like <code>^my-plugin:reviewer\$</code>
<code>PreCompact</code> , <code>PostCompact</code>	what triggered compaction	<code>manual</code> , <code>auto</code>
<code>SubagentStop</code>	agent type	same values as <code>SubagentStart</code>
<code>ConfigChange</code>	configuration source	<code>user_settings</code> , <code>project_settings</code> , <code>local_settings</code> , <code>policy_settings</code> , <code>skills</code>
<code>CwdChanged</code>	no matcher support	always fires on every directory change

FileChanged	literal filenames to watch (see FileChanged)	.envrc .env
StopFailure	error type	rate_limit , overloaded , authentication_failed , oauth_org_not_allowed , billing_error , invalid_request , model_not_found , server_error , max_output_tokens , unknown
InstructionsLoaded	load reason	session_start , nested_traversal , path_glob_match , include , compact
UserPromptExpansion	command name	your skill or command names
Elicitation	MCP server name	your configured MCP server names
ElicitationResult	MCP server name	same values as Elicitation
UserPromptSubmit , PostToolBatch , Stop , TeammateIdle , TaskCreated , TaskCompleted , WorktreeCreate , WorktreeRemove , MessageDisplay	no matcher support	always fires on every occurrence

The matcher runs against a field from the **JSON input** that Claude Code sends to your hook on stdin. For tool events, that field is `tool_name` . Each **hook event** section lists the full set of matcher values and the input schema for that event.

This example runs a linting script only when Claude writes or edits a file:

```
    "matcher": "Edit|Write",
    "hooks": [
      {
        "type": "command",
        "command": "/path/to/lint-check.sh"
      }
    ]
  }
]
```

If you add a `matcher` field to an event without matcher support, it is silently ignored.

For tool events, you can filter more narrowly by setting the `if` field on individual hook handlers.

`if` uses permission rule syntax to match against the tool name and arguments together, so

`"Bash(git *)"` runs when any subcommand of the Bash input matches `git *` and

`"Edit(*.ts)"` runs only for TypeScript files.

Match MCP tools

MCP server tools appear as regular tools in tool events (`PreToolUse` , `PostToolUse` , `PostToolUseFailure` , `PermissionRequest` , `PermissionDenied`), so you can match them the same way you match any other tool name.

MCP tools follow the naming pattern `mcp__<server>__<tool>` , for example:

- `mcp__memory__create_entities` : Memory server's create entities tool
- `mcp__filesystem__read_file` : Filesystem server's read file tool
- `mcp__github__search_repositories` : GitHub server's search tool

To match every tool from a server, append `.*` to the server prefix. The `.*` is required: a matcher like `mcp__memory` or `mcp__brave-search` contains only exact-match characters, so it is compared as an exact string and matches no tool.

- `mcp__memory__.*` matches all tools from the `memory` server
- `mcp__brave-search__.*` matches all tools from a server whose name contains a hyphen

matches every tool from that server. The `mcp__plugin__search__.*` form works on every version.

Tools from a **plugin-bundled MCP server** use a scoped server segment that includes the plugin name: `mcp__plugin_<plugin-name>_<server-name>__<tool>`. A matcher written against the bare server key never fires for these tools. For a plugin named `my-plugin` that bundles a server under the key `db`, a `query` tool appears as `mcp__plugin_my-plugin_db__query`, so the matcher for every tool from that server is `mcp__plugin_my-plugin_db__.*`. Use the same scoped tool name in a handler's `if` field. See **Plugin-provided MCP servers** for how the scoped name is built.

This example logs all memory server operations and validates write operations from any MCP server:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "mcp__memory__.*",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Memory operation initiated' >> ~/mcp-operations.log"
          }
        ]
      },
      {
        "matcher": "mcp__.*__write.*",
        "hooks": [
          {
            "type": "command",
            "command": "/home/user/scripts/validate-mcp-write.py"
          }
        ]
      }
    ]
  }
}
```

- **Command hooks** (`type: "command"`): run a shell command. Your script receives the event's **JSON input** on stdin and communicates results back through exit codes and stdout.
- **HTTP hooks** (`type: "http"`): send the event's JSON input as an HTTP POST request to a URL. The endpoint communicates results back through the response body using the same **JSON output format** as command hooks.
- **MCP tool hooks** (`type: "mcp_tool"`): call a tool on an already-connected **MCP server**. The tool's text output is treated like command-hook stdout.
- **Prompt hooks** (`type: "prompt"`): send a prompt to a Claude model for single-turn evaluation. The model returns its decision as JSON. See **Prompt-based hooks**.
- **Agent hooks** (`type: "agent"`): spawn a subagent that can use tools like Read, Grep, and Glob to verify conditions before returning a decision. Agent hooks are experimental and may change. See **Agent-based hooks**.

All matching hooks run in parallel. If you define the same handler in more than one settings file, it runs once. A plugin's or skill's copy of the same handler stays separate.

Handlers run in the current directory with Claude Code's environment. The `$CLAUDE_CODE_REMOTE` environment variable is set to `"true"` in remote web environments and not set in the local CLI. As of v2.1.199, `$CLAUDE_CODE_BRIDGE_SESSION_ID` is set to the **Remote Control** session ID while the local session has an active Remote Control connection.

Common fields

These fields apply to all hook types:

Field	Required	Description
<code>type</code>	yes	<code>"command"</code> , <code>"http"</code> , <code>"mcp_tool"</code> , <code>"prompt"</code> , or <code>"agent"</code>
<code>if</code>	no	Permission rule syntax to filter when this hook runs, such as <code>"Bash(git *)"</code> or <code>"Edit(*.ts)"</code> . The hook command only runs if the tool call matches the pattern. See the Bash matching table below for how Bash patterns evaluate against subcommands, <code>\$()</code> , and backticks. Only evaluated on tool events: <code>PreToolUse</code> , <code>PostToolUse</code> , <code>PostToolUseFailure</code> , <code>PermissionRequest</code> , and <code>PermissionDenied</code> . On other events, a hook with <code>if</code> set never runs. Uses the same syntax as permission rules

default to 30, and `MessageDisplay` lowers it to 10. `SessionEnd` hooks share a 1.5-second budget; if your settings set a longer per-hook `timeout`, Claude Code raises the budget to match, up to 60 seconds

<code>statusMessage</code>	no	Custom spinner message displayed while the hook runs
<code>once</code>	no	If <code>true</code> , Claude Code removes the hook after its first successful run. A run that fails, blocks with exit code 2, or times out leaves the hook in place, so it runs again on the next matching event. Only honored for hooks declared in <code>skill frontmatter</code> ; ignored in settings files and agent frontmatter

The `if` field holds exactly one permission rule. There is no `&&`, `||`, or list syntax for combining rules; to apply multiple conditions, define a separate hook handler for each.

In an `if` condition for a file tool, a single-segment directory pattern like `"Edit(src/**)"` matches only the `src` directory in the working directory and the files under it. To match a directory named `src` at any depth, write `"Edit(**/src/**)"`. Before v2.1.214, `"Edit(src/**)"` matched a directory named `src` at any depth under the working directory.

For Bash patterns, whether your hook command runs depends on the shape of the pattern and the Bash command Claude is invoking. Leading `VAR=value` assignments are stripped before matching.

<code>if</code> pattern	Bash command	Hook runs?	Why
<code>Bash(git *)</code>	<code>F00=bar git push</code>	yes	leading assignments are stripped; <code>git push</code> matches
<code>Bash(git *)</code>	<code>npm test && git push</code>	yes	each subcommand is checked; <code>git push</code> matches
<code>Bash(rm *)</code>	<code>echo \$(rm -rf /)</code>	yes	commands inside <code>\$()</code> and backticks are checked; <code>rm -rf /</code> matches
<code>Bash(rm *)</code>	<code>echo \$(date)</code>	no	no subcommand matches <code>rm *</code>
<code>Bash(git push *)</code>	<code>echo \$(date)</code>	yes	patterns that specify more than the command name run the hook anyway on <code>\$()</code> , backticks, or <code>\$VAR</code>

The filter also fails open, running your hook regardless of pattern, when the Bash command can't be parsed. Because the `if` filter is best-effort, use the **`permission system`** rather than a hook to

Field	Required	Description
<code>command</code>	yes	Shell command to execute. With <code>args</code> , the executable to spawn directly. See Exec form and shell form
<code>args</code>	no	Argument list. When present, <code>command</code> is resolved as an executable and spawned directly with <code>args</code> as the argument vector, with no shell involved. See Exec form and shell form
<code>async</code>	no	If <code>true</code> , runs in the background without blocking. See Run hooks in the background
<code>asyncRewake</code>	no	If <code>true</code> , runs in the background and wakes Claude on exit code 2. The hook's stderr, or stdout if stderr is empty, is shown to Claude as a system reminder so it can react to a long-running background failure
<code>shell</code>	no	Shell to use for this hook. Accepts <code>"bash"</code> or <code>"powershell"</code> . Defaults to <code>"bash"</code> , or to <code>"powershell"</code> on Windows when Git Bash isn't installed. Setting <code>"powershell"</code> runs the command via PowerShell on Windows. Does not require <code>CLAUDE_CODE_USE_POWERSHELL_TOOL</code> since hooks spawn PowerShell directly. Ignored when <code>args</code> is set

Exec form and shell form

A command hook runs as exec form when `args` is set, and shell form when `args` is omitted. Set `args` whenever the hook references a [path placeholder](#), since each element is passed as one argument with no quoting. Omit `args` when you need shell features like pipes or `&&`, or when neither concern applies.

Exec form runs when `args` is present. Claude Code resolves `command` as an executable on `PATH` and spawns it directly with `args` as the argument vector. There is no shell, so each `args` element is one argument exactly as written, and path placeholders like `${CLAUDE_PLUGIN_ROOT}` are substituted into `command` and into each `args` element as plain strings. Special characters such as apostrophes, `$`, and backticks pass through verbatim because there is no shell to interpret them. No shell tokenization happens on any platform.

Shell form runs when `args` is absent. The `command` string is passed to a shell: `sh -c` on macOS and Linux, Git Bash on Windows, or PowerShell when Git Bash isn't installed. Set the `shell` field to

and `.bat` shims that `npm`, `npx`, `eslint`, and other tools install in `node_modules/.bin` are not executables and can't be spawned without a shell. To run them in exec form, invoke the underlying script with `node` directly, for example `"command": "node", "args": ["${CLAUDE_PLUGIN_ROOT}/node_modules/eslint/bin/eslint.js"]`. The `node` plus script-path pattern works on every platform because `node.exe` is a real binary. To run a `.cmd` or `.bat` shim by name, use shell form.

This example runs a Node script bundled with a plugin. Exec form passes the resolved script path as one argument with no quoting:

```
{
  "type": "command",
  "command": "node",
  "args": ["${CLAUDE_PLUGIN_ROOT}/scripts/format.js", "--fix"]
}
```

The equivalent shell form needs quoting to handle paths with spaces or special characters:

```
{
  "type": "command",
  "command": "node \"${CLAUDE_PLUGIN_ROOT}\"/scripts/format.js --fix"
}
```

Both forms support the same **path placeholders**, and both export them as the environment variables `CLAUDE_PROJECT_DIR`, `CLAUDE_PLUGIN_ROOT`, and `CLAUDE_PLUGIN_DATA` on the spawned process, so a script can read `process.env.CLAUDE_PLUGIN_ROOT` regardless of how it was launched.

Plugin hooks additionally substitute `${user_config.*}` values, in exec form only: the value is substituted into `command` and into each `args` element as a plain string, so no shell re-parses it.

A shell-form plugin hook whose `command` references `${user_config.*}` fails with an **error** instead of running. To use an option value from a shell-form hook, read the

`$CLAUDE_PLUGIN_OPTION_<KEY>` environment variable, such as

`$CLAUDE_PLUGIN_OPTION_WEBHOOK_URL` for a `webhook_url` option, or set `args` to switch the hook to exec form. Before v2.1.207, shell-form plugin hook commands also substituted

`${user_config.*}`.

HTTP hook fields

In addition to the [common fields](#), HTTP hooks accept these fields:

Field	Required	Description
<code>url</code>	yes	URL to send the POST request to
<code>headers</code>	no	Additional HTTP headers as key-value pairs. Values support environment variable interpolation using <code>\$VAR_NAME</code> or <code>\${VAR_NAME}</code> syntax. Only variables listed in <code>allowedEnvVars</code> are resolved
<code>allowedEnvVars</code>	no	List of environment variable names that may be interpolated into header values. References to unlisted variables are replaced with empty strings. Required for any env var interpolation to work

Claude Code sends the hook's **JSON input** as the POST request body with `Content-Type: application/json`. The response body uses the same **JSON output format** as command hooks.

Error handling differs from command hooks; see [HTTP response handling](#).

This example sends `PreToolUse` events to a local validation service, authenticating with a token from the `MY_TOKEN` environment variable:

```

"matcher": "Bash",
"hooks": [
  {
    "type": "http",
    "url": "http://localhost:8080/hooks/pre-tool-use",
    "timeout": 30,
    "headers": {
      "Authorization": "Bearer $MY_TOKEN"
    },
    "allowedEnvVars": ["MY_TOKEN"]
  }
]
}
]
}
}

```

MCP tool hook fields

In addition to the [common fields](#), MCP tool hooks accept these fields:

Field	Required	Description
<code>server</code>	yes	Name of a configured MCP server. For a plugin-bundled server , this is the scoped name <code>plugin:<plugin-name>:<server-name></code> , such as <code>plugin:my-plugin:db</code> , not the bare server key. The server must already be connected; the hook never triggers an OAuth or connection flow
<code>tool</code>	yes	Name of the tool to call on that server
<code>input</code>	no	Arguments passed to the tool. String values support <code>\${path}</code> substitution from the hook's JSON input , such as <code>"\${tool_input.file_path}"</code>

Claude Code reads the tool's text content the same way it reads command-hook stdout, following the [parsing rule under exit code 0](#). If the named server is not connected, or the tool returns `isError: true`, the hook produces a non-blocking error and execution continues.

MCP tool hooks are available on every hook event once Claude Code has connected to your MCP servers. `SessionStart` and `Setup` typically fire before servers finish connecting, so hooks on

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "mcp_tool",
            "server": "my_server",
            "tool": "security_scan",
            "input": { "file_path": "${tool_input.file_path}" }
          }
        ]
      }
    ]
  }
}
```

Prompt and agent hook fields

In addition to the **common fields**, prompt and agent hooks accept these fields:

Field	Required	Description
<code>prompt</code>	yes	Prompt text to send to the model. Use <code>\$ARGUMENTS</code> as a placeholder for the hook input JSON. Escape with a backslash to include literal text: <code>\\$1.00</code> renders as <code>\$1.00</code>
<code>model</code>	no	Model to use for evaluation. Defaults to a fast model

Reference scripts by path

Use these placeholders to reference hook scripts relative to the project or plugin root, regardless of the working directory when the hook runs:

- `${CLAUDE_PROJECT_DIR}` : the project root where the session started. Claude Code also sets this variable in the environment of **studio MCP servers** and plugin LSP servers.

❗ **Worktrees are different.** If Claude enters a worktree during the session, Claude Code keeps `${CLAUDE_PROJECT_DIR}` where it was and passes the worktree path to your hooks a different way:

- `${CLAUDE_PROJECT_DIR}` **stays put:** it still points at the project root where the session started, so a command such as `${CLAUDE_PROJECT_DIR}/.claude/hooks/check-style.sh` still runs the script in the main checkout.
- `cwd` **follows Claude:** the `cwd` field in the hook's input JSON is the worktree root after Claude enters a worktree, and the new directory after Claude runs `cd`. Read it when a hook needs to know which directory Claude is working in.

Prefer exec form for any hook that references a path placeholder. In shell form, wrap each placeholder in double quotes.

Project scripts Plugin scripts

This example uses `${CLAUDE_PROJECT_DIR}` to run a style checker from the project's `.claude/hooks/` directory after any `Write` or `Edit` tool call:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/check-style.sh",
            "args": []
          }
        ]
      }
    ]
  }
}
```

- **Subagent hooks:** Claude Code runs them only while that subagent is running and removes them when it finishes. Claude Code converts a `Stop` hook here to `SubagentStop`, the event it fires when a subagent completes.
- **Skill hooks:** Claude Code registers them when you or Claude invoke the skill and keeps running them for the rest of the session, on turns after the skill's own turn as well. To have Claude Code remove a hook after its first successful run instead, set `once: true` on it.

All hook events are supported.

This skill defines a `PreToolUse` hook that runs a security validation script before each `Bash` command:

```
---
name: secure-operations
description: Perform operations with security checks
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/security-check.sh"
---
```

Subagents use the same format in their YAML frontmatter.

Frontmatter hooks in a project skill follow the same workspace trust rule as hooks in settings files. Claude Code registers them when you or Claude invoke the skill, including in a `-p` run in a folder you haven't trusted.

Frontmatter hooks in a project subagent run only after you accept the workspace trust dialog for the folder the agent file came from. A `-p` session doesn't count as accepting it. What runs before you trust a folder compares this with the settings-file rule, and the subagents page lists which scopes are exempt. Before v2.1.218, these hooks could run from folders you hadn't trusted.

came from, or inspect a hook's command, prompt, or URL.

The menu displays all five hook types: `command` , `prompt` , `agent` , `http` , and `mcp_tool` . Each hook is labeled with a `[type]` prefix and a source indicating where it was defined:

- `User Settings` : from `~/.claude/settings.json`
- `Project Settings` : from `.claude/settings.json`
- `Local Settings` : from `.claude/settings.local.json`
- `Plugin Hooks` : from a plugin's `hooks/hooks.json`
- `Session Hooks` : registered in memory for the current session

Selecting a hook opens a detail view showing its event, matcher, type, source file, and the full command, prompt, or URL. The menu is read-only: to add, modify, or remove hooks, edit the settings JSON directly or ask Claude to make the change.

Disable or remove hooks

To remove a hook, delete its entry from the settings JSON file.

To temporarily disable all hooks without removing them, set `"disableAllHooks": true` in your settings file. Claude Code reads the value left after [settings precedence](#) applies, so a

`"disableAllHooks": false` in a project's `.claude/settings.json` overrides a `true` in your user settings. To turn hooks off for one run whatever the project's settings say, pass `--settings '{"disableAllHooks": true}'` , which takes precedence over project and local settings. There is no way to disable an individual hook while keeping it in the configuration.

The `disableAllHooks` setting respects the managed settings hierarchy. If an administrator has configured hooks through managed policy settings, `disableAllHooks` set in user, project, or local settings can't disable those managed hooks. Only `disableAllHooks` set at the managed settings level can disable managed hooks. For the full reach of each level, see [disableAllHooks](#) .

Direct edits to hooks in settings files are normally picked up automatically by the file watcher.

through the HTTP response body. This section covers fields and behavior common to all events.

Each event's section under **Hook events** includes its specific input schema and decision control options.

On macOS and Linux, command hooks run in their own session without a controlling terminal. The hook process and any child processes can't open `/dev/tty` or send escape sequences directly to the Claude Code interface. Windows has no `/dev/tty`.

To surface a message to the user on any platform, return `systemMessage` in JSON output. Some events discard it or deliver it elsewhere, and each **event's section** says so. To trigger a desktop notification, set a window title, or ring the bell, return `terminalSequence` instead.

Common input fields

Hook events receive these fields as JSON, in addition to event-specific fields documented in each **hook event** section. For command hooks, this JSON arrives via stdin. For HTTP hooks, it arrives as the POST request body.

Field	Description
<code>session_id</code>	Current session identifier
<code>prompt_id</code>	UUID identifying the user prompt currently being processed. Matches the <code>prompt.id</code> <u>attribute on OpenTelemetry events</u> , so you can correlate hook output with telemetry for a single prompt. Absent until the first user input. Requires Claude Code v2.1.196 or later
<code>transcript_path</code>	Path to conversation JSON. The transcript file is written asynchronously and may lag the in-memory conversation, so it may not yet include the current turn's most recent messages when a hook fires. Hooks that need the final assistant text of the current turn should use <code>last_assistant_message</code> on <u>Stop</u> and <u>SubagentStop</u> instead of reading the transcript
<code>cwd</code>	Current working directory when the hook is invoked
<code>permission_mode</code>	Current permission mode : <code>"default"</code> , <code>"plan"</code> , <code>"acceptEdits"</code> , <code>"auto"</code> , <code>"dontAsk"</code> , or <code>"bypassPermissions"</code> . The mode labeled Manual arrives as <code>"default"</code> , never as <code>"manual"</code> , so scripts that match <code>"default"</code> keep working. Not all events receive this field. Check the JSON example in each <u>hook event</u> section

`line` `effort` field. Present for events that fire within a tool-use context, such as `PreToolUse` , `PostToolUse` , `Stop` , and `SubagentStop` , when the current model supports the effort parameter. The level is also available to hook commands and the Bash tool as the `$CLAUDE_EFFORT` environment variable.

<code>hook_event_name</code>	Name of the event that fired
------------------------------	------------------------------

When running with `--agent` or inside a subagent, two additional fields are included:

Field	Description
<code>agent_id</code>	Unique identifier for the subagent. Present only when the hook fires inside a subagent call. Use this to distinguish subagent hook calls from main-thread calls.
<code>agent_type</code>	Agent name (for example, <code>"Explore"</code> or <code>"security-reviewer"</code>). Present when the session uses <code>--agent</code> or the hook fires inside a subagent. For subagents, the subagent's type takes precedence over the session's <code>--agent</code> value. See SubagentStart for the values custom and plugin subagents report and how to write a matcher against a plugin-scoped name.

Only `SessionStart` hooks can receive a `model` field, and it is not guaranteed to be present. There is no `$CLAUDE_MODEL` environment variable. A hook process inherits the parent environment, so it can read `$ANTHROPIC_MODEL` if you set it in your shell, but that value doesn't change when you switch models with `/model` during a session. One set of variables is not inherited: Claude Code **removes `OTEL_*` exporter variables from every subprocess it spawns**, including hooks.

For example, a `PreToolUse` hook for a Bash command receives this on stdin:

```
"cwd": "/home/user/my-project",
"permission_mode": "default",
"hook_event_name": "PreToolUse",
"tool_name": "Bash",
"tool_input": {
  "command": "npm test",
  "description": "Run test suite",
  "timeout": 120000,
  "run_in_background": false
},
"tool_use_id": "toolu_01ABC123..."
}
```

The `tool_name`, `tool_input`, and `tool_use_id` fields are event-specific. Each hook event section documents the additional fields for that event.

Exit code output

The exit code from your hook command tells Claude Code whether the action should proceed, be blocked, or be ignored. The exit code doesn't act alone. Claude Code reads JSON output fields from stdout on every exit code, not just 0, and for events that use the standard decision model, a parsed object that passes schema validation takes effect alongside the code. Exit 2's block is the one outcome JSON can't override.

Two tables own the per-event exceptions: Exit code 2 behavior per event says what exit codes do for each event, and Decision control says which decision fields each event honors. Universal fields such as `systemMessage` work across most events and are listed in the JSON output table.

Exit code 0

Exit 0 means success, and is the intended exit code when you print JSON for structured control. For most events, stdout is written to the debug log but not shown in the transcript. The exceptions are `UserPromptSubmit`, `UserPromptExpansion`, and `SessionStart`, where Claude Code adds plain-text stdout as context that Claude can see and act on.

Whether Claude Code reads your stdout as JSON output or as plain text depends on its first character, ignoring leading whitespace:

For events that use the standard decision model, exit 0 with a parsed object that fails schema validation is a non-blocking error: the action proceeds, and the transcript shows a `<hook name> hook error` notice with the validation message. The same happens on any exit code other than 2, while **exit 2 still blocks**.

Stderr from a hook that exits 0 goes to the debug log only, never the transcript, and Claude never sees it. To read it yourself, enable **debug logging**. To surface a warning to Claude from a `PostToolUse` or `PostToolUseFailure` hook, exit 2 instead so **Claude sees the stderr** even though the tool already ran.

Exit code 2

Exit 2 means a blocking error. On **events that can block**, exit 2 blocks whether or not you print JSON: even a JSON `permissionDecision` of `"allow"` can't override it. Claude Code still reads any valid **JSON output** on stdout. On `Elicitation` and `ElicitationResult`, an exit-2 hook's `hookSpecificOutput` is ignored.

The blocking message is the reason from your JSON's blocking decision when it makes one, and your stderr text otherwise. What the block does varies by event: `PreToolUse` blocks the tool call, `UserPromptSubmit` rejects the prompt, and so on. **Exit code 2 behavior per event** lists the effect for every event, and each event's section says where the message goes.

A hook that exits 2 while printing JSON that fails **JSON output** schema validation still blocks: Claude Code uses stderr as the blocking reason and records the validation failure in the debug log. Before v2.1.214, Claude Code treated that combination as a non-blocking error and the action proceeded.

This script blocks `rm` commands by exiting 2 and leaves every other command to the normal permission flow:

```
if [[ "$command" == rm* ]]; then
  echo "Blocked: rm commands are not allowed" >&2
  exit 2 # Blocking error: tool call is prevented
fi

exit 0 # No decision: the normal permission flow applies
```

Other exit codes

Any other exit code doesn't block on its own for most hook events. What happens depends on your stdout:

- With a parsed object that passes schema validation, for events that use the standard decision model, Claude Code ignores the exit code and the JSON alone decides the outcome:
 - Each field the event supports is honored, including `permissionDecision` , `additionalContext` , `updatedInput` , and `systemMessage` , and the hook isn't reported as an error.
 - **Decision control** lists the decision fields per event; universal fields like `systemMessage` follow the **JSON output** table.
- With a parsed object that fails schema validation, for events that use the standard decision model, it's the same non-blocking error as **on exit 0**: the action proceeds, and the `<hook name> hook error` notice carries the validation message.
- With stdout that Claude Code **treats as plain text**, or with empty stdout, it's a non-blocking error for most hook events: the action proceeds, and the transcript shows a `<hook name> hook error` notice followed by the first line of stderr, prefixed with `Failed with non-blocking status code:` . To capture the full stderr, enable **debug logging**.

Events outside the standard decision model keep their own rows in the **per-event table**:

`WorktreeCreate` fails creation on any nonzero exit no matter what your JSON says, and events that discard hook output entirely, like `StopFailure` , ignore your JSON on every exit code, apart from side-effect fields like `terminalSequence` , which still fire.

A hook that can't start lands in the same non-blocking bucket. When the script path doesn't exist or isn't executable, the shell exits with a code like 127 and you see the same notice with the interpreter's message, for example `Failed with non-blocking status code: /bin/sh:`

⚠ For most hook events, exit code 2 is the only exit code that blocks through the code alone. Without valid JSON on stdout, Claude Code treats exit code 1 as a non-blocking error and proceeds with the action, even though 1 is the conventional Unix failure code. If your hook is meant to enforce a policy, use `exit 2`. The exception is `WorktreeCreate`, where any non-zero exit code aborts worktree creation.

Timeouts

Apart from a command hook you run with `async: true`, a `command`, `http`, or `mcp_tool` hook that reaches its `timeout` is canceled: Claude Code discards the hook's output, and the hook renders no decision. On `PreToolUse`, the two hook families differ:

- A timed-out `command`, `http`, or `mcp_tool` hook doesn't block the tool call. The call continues through the normal **permission flow**, so don't count on a stalled hook to act as a gate.
- An **Agent SDK callback hook** that exceeds its timeout **blocks the tool call**.

Exit code 2 behavior per event

Exit code 2 is the way a hook signals "stop, don't do this." The effect depends on the event, because some events represent actions that can be blocked (like a tool call that hasn't happened yet) and others represent things that already happened or can't be prevented.

Hook event	Can block?	What happens on exit 2
<code>PreToolUse</code>	Yes	Blocks the tool call
<code>PermissionRequest</code>	No	Exit code 2 isn't honored for this event and the permission flow proceeds unchanged. Deny through the decision object instead
<code>UserPromptSubmit</code>	Yes	Blocks prompt processing and erases the prompt
<code>UserPromptExpansion</code>	Yes	Blocks the expansion
<code>Stop</code>	Yes	Prevents Claude from stopping, continues the conversation
<code>SubagentStop</code>	Yes	Prevents the subagent from stopping
<code>TeammateIdle</code>	Yes	Prevents the teammate from going idle, so it continues working
<code>TaskCreated</code>	Yes	Rolls back the task creation
<code>TaskCompleted</code>	Yes	Prevents the task from being marked as completed

PostToolUse	No	Shows stderr to Claude; the tool already ran
PostToolUseFailure	No	Shows stderr to Claude; the tool already failed
PostToolBatch	Yes	Stops the agentic loop before the next model call
PermissionDenied	No	Exit code and stderr are ignored because the denial already occurred. Use JSON <code>hookSpecificOutput.retry: true</code> to tell the model it may retry; Claude Code ignores <code>retry: true</code> for <u>no-verdict denials</u>
Notification	No	Exit code and stderr are ignored
SubagentStart	No	Shows stderr to user only
SessionStart	No	Shows stderr to user only
Setup	No	Exit code and stderr are ignored
SessionEnd	No	Shows stderr to user only
CwdChanged	No	Shows stderr to user only
DirectoryAdded	No	Stderr goes to the debug log; the directory is already added
FileChanged	No	Shows stderr to user only
PreCompact	Yes	Blocks compaction
PostCompact	No	Shows stderr to user only
Elicitation	Yes	Denies the elicitation
ElicitationResult	Yes	Blocks the response (action becomes decline)
WorktreeCreate	Yes	Any non-zero exit code causes worktree creation to fail
WorktreeRemove	No	Failures are logged in debug mode only
InstructionsLoaded	No	Exit code is ignored
MessageDisplay	No	The original text is displayed

For `SessionStart` and `SubagentStart`, the exit code 2 stderr renders in the transcript as a `<hook name> hook error` notice, the same way a **non-blocking error** does. Claude doesn't see it, and the session or subagent proceeds. For `SubagentStart`, the notice appears in the subagent's own transcript, not in the parent conversation.

- **2xx with an empty body:** success, equivalent to exit code 0 with no output
- **2xx with a JSON object body:** parsed using the same JSON output schema as command hooks. A body that fails schema validation is a non-blocking error
- **2xx with any other body, such as plain text:** non-blocking error, handled the same as a non-2xx status. Claude Code doesn't add the text to Claude's context
- **Non-2xx status:** non-blocking error, execution continues
- **Connection failure:** non-blocking error, execution continues
- **Timeout:** the hook is canceled and renders no decision, and execution continues

Unlike command hooks, HTTP hooks can't signal a blocking error through status codes alone. To block a tool call or deny a permission, return a 2xx response with a JSON body containing the appropriate decision fields.

JSON output

Exit codes only let you block or stay silent, but JSON output gives you finer-grained control. Instead of exiting with code 2 to block, exit 0 and print a JSON object to stdout. Claude Code reads specific fields from that JSON to control behavior, including decision control for blocking, allowing, or escalating to the user.

❗ Choose one approach per hook: either use exit codes alone for signaling, or exit 0 and print JSON for structured control. If you mix them, exit 2 keeps its blocking effect, and Claude Code still reads the JSON fields, with the one elicitation exception noted under Exit code 2.

Your hook's stdout must contain only the JSON object. If your shell profile prints text on startup, it can interfere with JSON parsing. See Hook JSON has no effect in the troubleshooting guide.

Hook output strings, including `additionalContext`, `systemMessage`, and plain stdout, are capped at 10,000 characters. Output that exceeds this limit is saved to a file and replaced with a preview and file path, the same way a large valid Bash result is handled under Output limits.

The JSON object supports three kinds of fields:

- **Top-level** `decision` **and** `reason` are used by some events to block or provide feedback.
- `hookSpecificOutput` is a nested object for events that need richer control. It requires a `hookEventName` field set to the event name.

Field	Default	Description
<code>continue</code>	<code>true</code>	If <code>false</code> , Claude stops processing entirely after the hook runs. Takes precedence over any event-specific decision fields
<code>stopReason</code>	<code>none</code>	Message shown to the user when <code>continue</code> is <code>false</code> . Not shown to Claude
<code>suppressOutput</code>	<code>false</code>	Has no effect: Claude Code accepts the field but doesn't act on it. A successful hook's stdout is never shown in the transcript and is recorded in the debug log
<code>systemMessage</code>	<code>none</code>	Warning message shown to the user. In Agent SDK and <code>--output-format stream-json</code> output, it can arrive as an <code>SDKInformationalMessage</code>
<code>terminalSequence</code>	<code>none</code>	A terminal escape sequence for Claude Code to emit on your behalf, such as a desktop notification, window title, or bell. Restricted to OSC <code>0 / 1 / 2 / 9 / 99 / 777</code> and BEL. If the value contains anything outside the allowlist, the field is ignored. Use this instead of writing to <code>/dev/tty</code> , which is unavailable to hooks

To stop Claude entirely:

```
{ "continue": false, "stopReason": "Build failed, fix errors before continuing" }
```

For `PreToolUse` and `PostToolUse` hooks, the stop applies even when the tool call fails or completes while Claude is still streaming a response.

Emit terminal notifications

Hooks run without a controlling terminal, so writing escape sequences directly to `/dev/tty` fails. Instead, return the escape sequence in the `terminalSequence` field and Claude Code emits it for you through its own terminal write path. This is race-free, works inside tmux and GNU screen, and works on Windows where there is no `/dev/tty`.

taskbar progress

- OSC 99 : Kitty notifications
- OSC 777 : urxvt, Ghostty, and Warp notifications
- Bare BEL

Sequences may be terminated with BEL or with ST. Anything outside the allowlist, including CSI cursor and color sequences, OSC palette sequences, OSC 8 hyperlinks, OSC 52 clipboard writes, and OSC 1337, is rejected and the field is ignored.

Claude Code writes the sequence itself when it processes your hook's output, so the field works on events that discard `systemMessage` and `continue`, such as `Notification` and `StopFailure`. It has two limits:

- Claude Code writes the sequence only in an interactive session, and only while its interface is on screen. In non-interactive mode with the `-p` flag and in the Agent SDK, it ignores the field.
- A `WorktreeCreate` command hook can't return JSON, because Claude Code reads its stdout as the worktree path. An HTTP `WorktreeCreate` hook returns JSON and can include the field.

The example below fires a desktop notification from a `Notification` hook. The escape sequence is built with `printf` octal escapes so the control bytes never appear on the shell command line, and `jq -n --arg` builds the JSON output so quotes, backslashes, and newlines in the notification message are escaped correctly:

```
#!/bin/bash
# Notification hook: ping the desktop when Claude Code needs attention.
input=$(cat)
title="Claude Code"
body=$(jq -r '.message // "Needs your attention"' <<<"$input")
seq=$(printf '\033]777;notify;%s;%s\007' "$title" "$body")
jq -nc --arg seq "$seq" '{terminalSequence: $seq}'
```

The `{ "terminalSequence": "..."` shape is the same from any shell or language.

chat message in the interface.

Return `additionalContext` inside `hookSpecificOutput` alongside the event name:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PostToolUse",
    "additionalContext": "This file is generated. Edit src/schema.ts and run `bun gene
  }
}
```

Where the reminder appears depends on the event:

- **SessionStart** and **SubagentStart**: at the start of the conversation, before the first prompt
- **UserPromptSubmit** and **UserPromptExpansion**: alongside the submitted prompt
- **PreToolUse**, **PostToolUse**, **PostToolUseFailure**, and **PostToolBatch**: next to the tool result
- **Stop** and **SubagentStop**: at the end of the turn. The conversation continues so Claude can act on the feedback. See **Stop decision control**

When several hooks return `additionalContext` for the same event, Claude receives all of the values. If a value exceeds 10,000 characters, Claude Code writes the full text to a file in the session directory and passes Claude the file path with a short preview instead.

Use `additionalContext` for information Claude should know about the current state of your environment or the operation that just ran:

- **Environment state**: the current branch, deployment target, or active feature flags
- **Conditional project rules**: which test command applies to the file just edited, which directories are read-only in this worktree
- **External data**: open issues assigned to you, recent CI results, content fetched from an internal service

For instructions that never change, prefer **CLAUDE.md**. It loads without running a script and is the standard place for static project conventions.

Claude Code saves the injected text in the session transcript. For mid-session events like `PostToolUse` or `UserPromptSubmit`, when you resume with `--continue` or `--resume`, Claude Code replays the saved text rather than re-running the hook for past turns, so values like timestamps or commit SHAs become stale. `SessionStart` hooks run again on resume with `source` set to `"resume"`, or `"fork"` if you added `--fork-session`, so they can refresh their context.

Decision control

Not every event supports blocking or controlling behavior through JSON. The events that do each use a different set of fields to express that decision. Use this table as a quick reference before writing a hook:

Events	Decision pattern	Key fields
UserPromptSubmit, UserPromptExpansion, PostToolUse, PostToolUseFailure, PostToolBatch, Stop, SubagentStop, ConfigChange, PreCompact	Top-level <code>decision</code>	<code>decision: "block"</code> , <code>reason</code> . Stop and SubagentStop also accept <code>hookSpecificOutput.additionalContext</code> for <u>non-error feedback that continues the conversation</u>
TeammateIdle, TaskCompleted	Exit code or <code>continue: false</code>	Exit code 2 blocks the action with stderr feedback. JSON <code>{"continue": false, "stopReason": "..."} </code> also stops the teammate entirely, matching <code>Stop</code> hook behavior; <u>TaskCompleted ignores it when the TaskUpdate tool triggered the event</u>
TaskCreated	Exit code or top-level <code>decision</code>	Exit code 2 or <code>decision: "block"</code> <u>cancels the task</u> and returns the message to Claude. <code>continue: false</code> is ignored
PreToolUse	<code>hookSpecificOutput.t</code>	<code>permissionDecision</code> (allow/deny/ask/defer), <code>permissionDecisionReason</code>
PermissionRequest	<code>hookSpecificOutput.t</code>	<code>decision.behavior</code> (allow/deny)
PermissionDenied	<code>hookSpecificOutput.t</code>	<code>retry: true</code> tells the model it may retry the denied tool call; Claude Code ignores it for <u>no-verdict denials</u>
WorktreeCreate	path return	Command hook prints path on stdout; HTTP hook returns <code>hookSpecificOutput.worktreePath</code> . Hook failure or missing path fails creation

	t	(form field values override)
MessageDisplay	hookSpecificOutput t	displayContent replaces the displayed text on screen. Display-only: the transcript and what Claude sees keep the original
SessionStart, SubagentStart	Context only	hookSpecificOutput.additionalContext adds context for Claude. SessionStart also accepts initialUserMessage , watchPaths , sessionTitle ,and reloadSkills . No blocking or decision control
Setup, WorktreeRemove, Notification, SessionEnd, PostCompact, InstructionsLoaded, StopFailure, CwdChanged, DirectoryAdded, FileChanged	None	No decision control. Used for side effects like logging or cleanup

A few events can also rewrite content rather than only allow or block it:

- PreToolUse : updatedInput directly under hookSpecificOutput replaces a tool’s arguments before it runs. See [PreToolUse decision control](#)
- PermissionRequest : updatedInput inside the decision object. See [PermissionRequest decision control](#)
- PostToolUse : updatedToolOutput replaces the tool’s result. See [PostToolUse decision control](#)
- UserPromptSubmit : can’t replace the prompt; it only injects additionalContext alongside it

For redaction or transformation use cases, intercept at PreToolUse for outbound tool inputs and PostToolUse for inbound tool results.

Here are examples of each pattern in action:

Top-level decision PreToolUse PermissionRequest

The only value for decision is "block" . To allow the action to proceed, omit decision from your JSON, or exit 0 without any JSON at all:

For extended examples including Bash command validation, prompt filtering, and auto-approval scripts, see [What you can automate](#) in the guide and the [Bash command validator reference implementation](#).

Hook events

Each event corresponds to a point in Claude Code's lifecycle where hooks can run. The sections below are ordered to match the lifecycle: from session setup through the agentic loop to session end. Each section describes when the event fires, what matchers it supports, the JSON input it receives, and how to control behavior through output.

SessionStart

Runs when Claude Code starts a new session or resumes an existing session. Useful for loading development context like existing issues or recent changes to your codebase, or setting up environment variables. For static context that doesn't require a script, use [CLAUDE.md](#) instead.

SessionStart runs on every session, so keep these hooks fast. Only `type: "command"` and `type: "mcp_tool"` hooks are supported.

The matcher value corresponds to how the session was initiated:

Matcher	When it fires
<code>startup</code>	New session
<code>resume</code>	<code>--resume</code> , <code>--continue</code> , or <code>/resume</code>
<code>clear</code>	<code>/clear</code>
<code>compact</code>	Auto or manual compaction
<code>fork</code>	A new session forked from an existing one: <code>--fork-session</code> with <code>--resume</code> or <code>--continue</code> , the <code>/fork</code> background copy, or <code>/branch</code>

Before v2.1.214, forked sessions reported source `"resume"` .

Field	Description
<code>source</code>	How the session started: <code>"startup"</code> for new sessions, <code>"resume"</code> for resumed sessions, <code>"clear"</code> after <code>/clear</code> , <code>"compact"</code> after compaction, or <code>"fork"</code> for a new session forked from an existing one
<code>model</code>	The active model identifier. It can be omitted, for example after <code>/clear</code> or when a session is restored through conversation recovery, so check for the field before reading it
<code>agent_type</code>	The agent name, present when you start Claude Code with <code>claude --agent <name></code>
<code>session_title</code>	The current session title if one is already set, for example via <code>--name</code> or <code>/rename</code> . A hook that emits <code>sessionTitle</code> can check <code>session_title</code> first to avoid overwriting a title the user set explicitly

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9b...",
  "cwd": "/Users/...",
  "hook_event_name": "SessionStart",
  "source": "startup",
  "model": "claude-sonnet-5"
}
```

SessionStart decision control

Claude Code adds stdout it treats as plain text to Claude's context. In addition to the JSON output fields available to all hooks, you can return these event-specific fields:

Field	Description
<code>additionalContext</code>	String added to Claude's context at the start of the conversation, before the first prompt. See <u>Add context for Claude</u> for how the text is delivered and what to put in it
<code>initialUserMessage</code>	String used as the first user message of the session. Applies in <u>non-interactive mode</u> with the <code>-p</code> flag, where it becomes the first turn even if no prompt is provided. If a prompt is provided, it follows as the next turn. Unlike <code>additionalContext</code> , which attaches to an existing turn, this creates the turn
<code>sessionTitle</code>	Sets the session title, with the same effect as <code>/rename</code> . Use to name sessions automatically from the launch folder, git branch, or worktree name. Applies when <code>source</code> is <code>"startup"</code> , <code>"resume"</code> , or <code>"fork"</code> ; ignored on <code>"clear"</code> and <code>"compact"</code>

session, starting with the first prompt

```
{
  "hookSpecificOutput": {
    "hookEventName": "SessionStart",
    "additionalContext": "Current branch: feat/auth-refactor\nUncommitted changes: src
    "sessionTitle": "auth-refactor"
  }
}
```

Since plain stdout already reaches Claude for this event, a hook that only loads context can print to stdout directly without building JSON. Use the JSON form when you need to combine context with other fields such as `sessionTitle`.

Use `reloadSkills` when a `SessionStart` hook installs or updates skills. Skill discovery normally runs before `SessionStart` hooks finish, so files the hook writes into `~/.claude/skills/` or `.claude/skills/` would otherwise only appear in the next session. This example syncs a shared skills repository and requests the re-scan:

```
#!/bin/bash

git -C ~/.claude/skills/team-skills pull --quiet 2>/dev/null || \
  git clone --quiet https://git.example.com/your-org/team-skills.git ~/.claude/skills/

echo '{"hookSpecificOutput": {"hookEventName": "SessionStart", "reloadSkills": true}}'
```

The repository URL is a placeholder; replace it with your own skills repository. With the placeholder, the clone fails and prints a `fatal:` message to stderr. Stderr from a `SessionStart` hook that exits 0 is informational only, so the `reloadSkills` request still applies.

Persist environment variables

`SessionStart` hooks have access to the `CLAUDE_ENV_FILE` environment variable, which provides a file path where you can persist environment variables for subsequent Bash commands.

```
if [ -n "$CLAUDE_ENV_FILE" ]; then
  echo 'export NODE_ENV=production' >> "$CLAUDE_ENV_FILE"
  echo 'export DEBUG_LOG=true' >> "$CLAUDE_ENV_FILE"
  echo 'export PATH="$PATH:./node_modules/.bin"' >> "$CLAUDE_ENV_FILE"
fi

exit 0
```

To capture all environment changes from setup commands, compare the exported variables before and after:

```
#!/bin/bash

ENV_BEFORE=$(export -p | sort)

# Run your setup commands that modify the environment
source ~/.nvm/nvm.sh
nvm use 20

if [ -n "$CLAUDE_ENV_FILE" ]; then
  ENV_AFTER=$(export -p | sort)
  comm -13 <(echo "$ENV_BEFORE") <(echo "$ENV_AFTER") >> "$CLAUDE_ENV_FILE"
fi

exit 0
```

❗ `CLAUDE_ENV_FILE` is available for [SessionStart](#), [Setup](#), [CwdChanged](#), and [FileChanged](#) hooks. Other hook types don't have access to this variable.

Setup

Fires only when you launch Claude Code with `--init-only`, or with `--init` or `--maintenance` in **non-interactive mode** with the `-p` flag. It doesn't fire on normal startup. Use it for one-time dependency installation or scheduled cleanup that you trigger explicitly from CI or scripts, separate from normal session startup. For per-session initialization, use [SessionStart](#) instead.

`maintenance``claude -p --maintenance`

When you run `claude --init-only`, Claude Code runs Setup hooks and `SessionStart` hooks with the `startup` matcher, then exits without starting a conversation.

When you start or continue a conversation with `-p`, you also need to supply a prompt, as an argument or piped on stdin. You can skip the prompt when a `SessionStart` hook supplies `initialUserMessage` or when you resume a session with a **deferred tool call**.

On success, `--init-only` prints nothing to the terminal. To confirm the hooks ran, start with `claude --debug-file <path> --init-only`, replacing `<path>` with a log file location, and check the log for the Setup and SessionStart hook entries.

Because Setup doesn't fire on every launch, a plugin that needs a dependency installed can't rely on Setup alone. The practical pattern is to check for the dependency on first use and install on miss, for example a hook or skill that tests for `${CLAUDE_PLUGIN_DATA}/node_modules` and runs `npm install` if absent. See the **persistent data directory** for where to store installed dependencies. If you distribute your plugin through a marketplace, you may not need this pattern: Claude Code **installs eligible Node.js package dependencies automatically** when it caches the plugin.

Setup input

In addition to the **common input fields**, Setup hooks receive a `trigger` field set to either `"init"` or `"maintenance"` :

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9b",
  "cwd": "/Users/...",
  "hook_event_name": "Setup",
  "trigger": "init"
}
```

appear in the run's output only as `hook_response` **events** when you launch with `--output-format stream-json --verbose` .

Setup hooks have access to `CLAUDE_ENV_FILE` . Variables written to that file persist into subsequent Bash commands for the session, just as in [SessionStart hooks](#). Only `type: "command"` and `type: "mcp_tool"` hooks are supported.

InstructionsLoaded

Fires when a `CLAUDE.md` or `.claude/rules/*.md` file is loaded into context. This event fires at session start for eagerly-loaded files and again later when files are lazily loaded, for example when Claude accesses a subdirectory that contains a nested `CLAUDE.md` or when conditional rules with `paths: frontmatter` match. The hook doesn't support blocking or decision control. It runs asynchronously for observability purposes.

The matcher runs against `load_reason` . For example, use `"matcher": "session_start"` to fire only for files loaded at session start, or `"matcher": "path_glob_match|nested_traversal"` to fire only for lazy loads.

InstructionsLoaded input

In addition to the [common input fields](#), InstructionsLoaded hooks receive these fields:

Field	Description
<code>file_path</code>	Absolute path to the instruction file that was loaded
<code>memory_type</code>	Scope of the file: <code>"User"</code> , <code>"Project"</code> , <code>"Local"</code> , or <code>"Managed"</code>
<code>load_reason</code>	Why the file was loaded: <code>"session_start"</code> , <code>"nested_traversal"</code> , <code>"path_glob_match"</code> , <code>"include"</code> , or <code>"compact"</code> . The <code>"compact"</code> value fires when instruction files are re-loaded after a compaction event
<code>globs</code>	Path glob patterns from the file's <code>paths: frontmatter</code> , if any. Present only for <code>path_glob_match</code> loads
<code>trigger_file_path</code>	Path to the file whose access triggered this load, for lazy loads
<code>parent_file_path</code>	Path to the parent instruction file that included this one, for <code>include</code> loads

```
"hook_event_name": "InstructionsLoaded",
"file_path": "/Users/my-project/CLAUDE.md",
"memory_type": "Project",
"load_reason": "session_start"
}
```

InstructionsLoaded decision control

InstructionsLoaded hooks have no decision control. They can't block or modify instruction loading. Claude Code discards their **JSON output fields**, such as `systemMessage` and `continue`. Use this event for audit logging, compliance tracking, or observability.

UserPromptSubmit

Runs when the user submits a prompt, before Claude processes it. This allows you to add additional context based on the prompt/conversation, validate prompts, or block certain types of prompts.

`UserPromptSubmit` hooks have a default timeout of 30 seconds for `command`, `http`, and `mcp_tool` types, shorter than the 600-second default for those types on most other events. Because this hook runs before every prompt and blocks model processing until it completes, a stuck hook stalls the session. If your hook needs more time, set the `timeout` field in the hook entry.

Apart from a command hook you run with `async: true`, a `UserPromptSubmit` command, HTTP, or MCP tool hook that reaches its timeout is canceled and its output, including any `additionalContext`, is discarded. The prompt still reaches Claude without that context. The transcript shows a notice naming the hook, the timeout that fired, and that the output was discarded.

An **Agent SDK callback hook** on `UserPromptSubmit` that reaches its timeout blocks the prompt with a message naming the hook and the timeout, because a callback there can be acting as a policy gate that must not fail open. The session continues. Before v2.1.208, a callback timeout on that event ended the turn with an execution error.

UserPromptSubmit input

In addition to the **common input fields**, `UserPromptSubmit` hooks receive the `prompt` field containing the text the user submitted.

```
"permission_mode": "default",
"hook_event_name": "UserPromptSubmit",
"prompt": "Write a function to calculate the factorial of a number"
}
```

UserPromptSubmit decision control

`UserPromptSubmit` hooks can control whether a user prompt is processed and add context. All **JSON output fields** are available.

There are two ways to add context to the conversation on exit code 0:

- **Plain text stdout:** Claude Code adds stdout it **treats as plain text** to Claude's context
- **JSON with `additionalContext`:** use the JSON format below for more control. The `additionalContext` field is added as context

Neither channel produces a visible transcript entry. Plain stdout and the `additionalContext` value are each injected as a system reminder that starts with the hook's name; Claude reads both. To confirm delivery, check the **debug log**.

To block a prompt, return a JSON object with `decision` set to `"block"` :

Field	Description
<code>decision</code>	<code>"block"</code> prevents the prompt from being processed and erases it from context. Omit to allow the prompt to proceed
<code>reason</code>	Shown to the user when <code>decision</code> is <code>"block"</code> . Not added to context
<code>additionalContext</code>	String added to Claude's context alongside the submitted prompt. See <u>Add context for Claude</u>
<code>sessionTitle</code>	Sets the session title. Use to name sessions automatically based on the prompt content
<code>suppressOriginalPrompt</code>	If <code>true</code> when <code>decision</code> is <code>"block"</code> , omits the original prompt text from the block message shown to the user

A hook that blocks by exiting 2 routes the same way as `reason` : the block message shows the stderr text to the user, and it isn't added to context.

```
"hookEventName": "UserPromptSubmit",
"additionalContext": "My additional context here",
"sessionTitle": "My session title"
}
}
```

UserPromptExpansion

Runs when a user-typed command expands into a prompt before reaching Claude. Use this to block specific commands from direct invocation, inject context for a particular skill, or log which commands users invoke. For example, a hook matching `deploy` can block `/deploy` unless an approval file is present, or a hook matching a review skill can append the team's review checklist as `additionalContext` .

This event covers the path `PreToolUse` doesn't: a `PreToolUse` hook matching the `Skill` tool fires only when Claude calls the tool, but typing `/skillname` directly bypasses `PreToolUse` . `UserPromptExpansion` fires on that direct path.

Matches on `command_name` . Leave the matcher empty to fire on every prompt-type command.

UserPromptExpansion input

In addition to the **common input fields**, `UserPromptExpansion` hooks receive `expansion_type` , `command_name` , `command_args` , `command_source` , and the original `prompt` string. The `expansion_type` field is `slash_command` for skill and custom commands, or `mcp_prompt` for MCP server prompts.

```
"permission_mode": "default",
"hook_event_name": "UserPromptExpansion",
"expansion_type": "slash_command",
"command_name": "example-skill",
"command_args": "arg1 arg2",
"command_source": "plugin",
"prompt": "/example-skill arg1 arg2"
}
```

UserPromptExpansion decision control

`UserPromptExpansion` hooks can block the expansion or add context. All **JSON output fields** are available.

Field	Description
<code>decision</code>	<code>"block"</code> prevents the command from expanding. Omit to allow it to proceed
<code>reason</code>	Shown to the user when <code>decision</code> is <code>"block"</code>
<code>additionalContext</code>	String added to Claude's context alongside the expanded prompt. See <u>Add context for Claude</u>

A hook that blocks by exiting 2 routes the same way as `reason` : the block message shows the stderr text to the user.

```
{
  "decision": "block",
  "reason": "This slash command is not available",
  "hookSpecificOutput": {
    "hookEventName": "UserPromptExpansion",
    "additionalContext": "Additional context for this expansion"
  }
}
```

MessageDisplay

Runs while an assistant message streams to the screen. Claude Code displays the message in increments: each time a batch of newly completed lines is ready to render, the hook runs once with

- strip markdown for a minimal display
- transform the text an Agent SDK application shows its users
- redact API keys or internal hostnames from Claude's responses

Claude Code holds each batch until your hook returns, so keep the hook fast. If the hook fails or times out, Claude Code displays the original text. The default timeout for this event is 10 seconds; if your hook needs more time, set the `timeout` field in the hook entry.

MessageDisplay is display-only: the replacement text changes only what is rendered on screen. The transcript and what Claude sees keep the original text, so Claude never sees the replacement, and verbose mode shows the original. The hook receives assistant message text only, so tool results and the text you type render unchanged.

MessageDisplay doesn't support matchers and fires for every assistant message that streams text; messages with no text, such as tool-call-only responses, don't trigger it.

In non-interactive runs, including Agent SDK queries and `claude -p`, MessageDisplay runs once per assistant message instead of once per batch of lines. The single call arrives after the message completes and carries the full message text: `index` is `0`, `final` is `true`, and `delta` holds the entire message. A hook that collects the `delta` text for each message receives the same total text in both modes.

MessageDisplay input

In addition to the **common input fields**, MessageDisplay hooks receive identifiers for the turn and message, the position of this call within the message, and the new text in `delta`. Batch boundaries depend on how the text streams, so use `index` and `final` to track progress through a message rather than expecting lines to be grouped a particular way.

Field	Description
<code>turn_id</code>	UUID of the current turn
<code>message_id</code>	UUID of the assistant message being displayed. Stable across every batch of the same message. This is not the API <code>msg_...</code> id, so it can't be correlated with transcript message ids
<code>index</code>	Zero-based index of this batch within the message

batch's delta is empty when the message ends on a newline, so treat `final`, not a non-empty delta, as the end-of-message signal. In Agent SDK and `claude -p` runs, the single call carries the entire message

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../transcript.jsonl",
  "cwd": "/Users/my-project",
  "hook_event_name": "MessageDisplay",
  "turn_id": "0c9e6a2f-7d41-4f4e-9a15-3f4f7c2b8d10",
  "message_id": "5b2a9c8e-1f63-4d8a-b7c4-9e0d2a6f1c3b",
  "index": 0,
  "final": false,
  "delta": "Here is the plan:\n"
```

MessageDisplay output

In addition to the **JSON output fields** available to all hooks, MessageDisplay hooks can return `displayContent` to replace the delta on screen:

Field	Description
<code>displayContent</code>	Text displayed in place of the delta. Omit it to display the original

MessageDisplay hooks have no decision control. They can't block the message or change what is stored in the transcript or sent to Claude. Claude Code acts on `displayContent` from their JSON output and discards `systemMessage` and `continue`.

This example strips markdown formatting from Claude's responses for a plain-text display. The script reads each batch from stdin, removes bold markers and inline code backticks from `delta`, and returns the result as `displayContent`.

macOS/Linux **Windows (PowerShell)**

Register a command hook for the event in your settings file:

```
    "hooks": [
      {
        "type": "command",
        "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/plain-display.sh",
        "args": []
      }
    ]
  }
}
```

Save this script to `.claude/hooks/plain-display.sh` in your project and make it executable with `chmod +x` :

```
#!/bin/bash
jq '{hookSpecificOutput: {hookEventName: "MessageDisplay", displayContent: (.delta | c
```

Batches with no markdown pass through unchanged. If the script fails, for example because `jq` is missing, Claude Code displays the original text and notes the failure only in debug output, not in the session.

PreToolUse

Runs after Claude creates tool parameters and before processing the tool call. Matches on any tool name except `EndConversation` : built-in tools such as `Bash` , `PowerShell` , `Edit` , `Write` , `Read` , `Glob` , `Grep` , `Agent` , `Workflow` , `WebFetch` , `WebSearch` , `AskUserQuestion` , and `ExitPlanMode` , and any MCP tool names.

To run a hook when a specific file changes on disk, whatever wrote it, use FileChanged instead of matching file-editing tools by name. Unlike `PreToolUse`, Claude Code runs `FileChanged` hooks after the change, and they have no decision control, so they can't block the write.

PreToolUse also doesn't fire for EndConversation .

Use PreToolUse decision control to allow, deny, ask, or defer the tool call.

An Agent SDK callback hook on `PreToolUse` that exceeds its timeout blocks the tool call, and Claude receives an error result naming the timeout. An explicit deny returned by another hook still takes precedence.

PreToolUse input

In addition to the common input fields, PreToolUse hooks receive `tool_name` , `tool_input` , and `tool_use_id` .

For the file tools `Write` , `Edit` , and `Read` , `tool_input.file_path` is always absolute:

- Claude Code expands `~` and relative paths before hooks run, so a hook that matches on paths can't be bypassed via `~` or a relative spelling of the same path
- On Windows, the path arrives with backslash separators, even when your hook runs under Git Bash where `$PWD` looks like `/c/project`
- A comparison written with forward slashes, such as a `/src/` check, never matches a backslash path, and the tool call proceeds as if the hook had nothing to block
- Normalize separators before comparing: `FILE_PATH="${FILE_PATH//\\//}"` in Bash, or `file_path.replace("\\", "/")` in Python, then match a path segment such as `/src/` rather than anchoring with `^` , since the path is absolute

A `Write` call on Windows delivers:

```
{
  "hook_event_name": "PreToolUse",
  "tool_name": "Write",
  "tool_input": {
    "file_path": "C:\\project\\src\\index.ts",
    "content": "..."
  },
  ...
}
```

Field	Type	Example	Description
<code>command</code>	string	<code>"npm test"</code>	The shell command to execute
<code>description</code>	string	<code>"Run test suite"</code>	Optional description of what the command does
<code>timeout</code>	number	<code>120000</code>	Optional timeout in milliseconds. Values above the maximum are reduced to the maximum rather than rejected
<code>run_in_background</code>	boolean	<code>false</code>	Whether to run the command in background

PowerShell

Executes PowerShell commands. See the [PowerShell tool](#) for availability by platform.

The fields match the Bash tool, with the command string in `command` :

Field	Type	Example	Description
<code>command</code>	string	<code>"Get-ChildItem - Recurse"</code>	The PowerShell command to execute
<code>description</code>	string	<code>"List files recursively"</code>	Optional description of what the command does
<code>timeout</code>	number	<code>120000</code>	Optional timeout in milliseconds
<code>run_in_background</code>	boolean	<code>false</code>	Whether to run the command in background

Match `Bash|PowerShell` in hooks that inspect shell commands, so they cover both tools:

- On Windows, wherever the PowerShell tool is enabled, Claude treats PowerShell as the primary shell and routes shell commands through it.
- On Windows without Git Bash, the tool is enabled automatically and Claude Code doesn't register the Bash tool at all.
- A hook that matches only `Bash` never fires there.

Field	Type	Example	Description
<code>file_path</code>	string	<code>"/path/to/file.txt"</code>	Absolute path to the file to write
<code>content</code>	string	<code>"file content"</code>	Content to write to the file

Edit

Replaces a string in an existing file.

Field	Type	Example	Description
<code>file_path</code>	string	<code>"/path/to/file.txt"</code>	Absolute path to the file to edit
<code>old_string</code>	string	<code>"original text"</code>	Text to find and replace
<code>new_string</code>	string	<code>"replacement text"</code>	Replacement text
<code>replace_all</code>	boolean	<code>false</code>	Whether to replace all occurrences

Read

Reads file contents.

Field	Type	Example	Description
<code>file_path</code>	string	<code>"/path/to/file.txt"</code> <code>"</code>	Absolute path to the file to read
<code>offset</code>	number	<code>10</code>	Optional line number to start reading from
<code>limit</code>	number	<code>50</code>	Optional number of lines to read

Glob

Finds files matching a glob pattern.

Field	Type	Example	Description
<code>pattern</code>	string	<code>"**/*.ts"</code>	Glob pattern to match files against

Grep

Searches file contents with regular expressions.

Field	Type	Example	Description
<code>pattern</code>	string	<code>"TODO.*fix"</code>	Regular expression pattern to search for
<code>path</code>	string	<code>"/path/to/dir"</code>	Optional file or directory to search in
<code>glob</code>	string	<code>"*.ts"</code>	Optional glob pattern to filter files
<code>output_mode</code>	string	<code>"content"</code>	<code>"content"</code> , <code>"files_with_matches"</code> , or <code>"count"</code> . Defaults to <code>"files_with_matches"</code>
<code>-i</code>	boolean	<code>true</code>	Case insensitive search
<code>multiline</code>	boolean	<code>false</code>	Enable multiline matching

WebFetch

Fetches and processes web content.

Field	Type	Example	Description
<code>url</code>	string	<code>"https://example.com/api"</code>	URL to fetch content from
<code>prompt</code>	string	<code>"Extract the API endpoints"</code>	Prompt to run on the fetched content

WebSearch

Searches the web.

Field	Type	Example	Description
<code>query</code>	string	<code>"react hooks best practices"</code>	Search query
<code>allowed_domains</code>	array	<code>["docs.example.com"]</code>	Optional: only include results from these domains

Agent

Spawns a subagent.

Field	Type	Example	Description
<code>prompt</code>	string	<code>"Find all API endpoints"</code>	The task for the agent to perform
<code>description</code>	string	<code>"Find API endpoints"</code>	Short description of the task
<code>subagent_type</code>	string	<code>"Explore"</code>	Type of specialized agent to use
<code>model</code>	string	<code>"sonnet"</code>	Optional model alias to override the default

When a foreground Agent call completes, your PostToolUse hook receives the subagent's final text and run telemetry in `tool_response`. Read these fields to inspect the run; for token and cost rollups across subagents, use the token and cost counters filtered to `query_source` `"subagent"`, since `totalTokens` and `usage` cover the final request only:

Field	Type	Example	Description
<code>status</code>	string	<code>"completed"</code>	<code>"completed"</code> for foreground subagents, <code>"async_launched"</code> for background subagents. As of v2.1.198, subagents run in the background by default, so an omitted <code>run_in_background</code> also produces <code>"async_launched"</code>
<code>agentId</code>	string	<code>"a4d2c8f1e0b3a297"</code>	Identifier for the subagent run
<code>content</code>	array	<code>[{"type": "text", "text": "Found 12 endpoints..."}]</code>	The subagent's final text blocks
<code>resolvedModel</code>	string	<code>"claude-sonnet-4-5"</code>	Model the subagent started on, which may differ from the requested model. Requires Claude Code v2.1.174 or later
<code>modelsUsed</code>	array	<code>["claude-sonnet-4-5", "claude-haiku-4-5"]</code>	Models used in order, with consecutive repeats collapsed; set only when the model was swapped mid-run. Requires Claude Code v2.1.212 or later

<code>totalDurationMs</code>	number	<code>48211</code>	Wall-clock duration of the subagent run
<code>totalToolUseCount</code>	number	<code>7</code>	Count of tool calls the subagent made
<code>usage</code>	object	<code>{"input_tokens": 8320, ...}</code>	Per-type token breakdown of the final API request: <code>input_tokens</code> , <code>output_tokens</code> , <code>cache_creation_input_tokens</code> , <code>cache_read_input_tokens</code>

For background subagents, the tool returns when the task moves to the background, so `tool_response` carries no usage fields: a background launch returns immediately, and a foreground task that Claude Code backgrounds mid-run returns at that transition. It has `status: "async_launched"` , `agentId` , `description` , `prompt` , `outputFile` , and `resolvedModel` .

On a `completed` response, `resolvedModel` names the model the subagent started on, which can differ from the `model` value in `tool_input` , such as when `availableModels` or another override applies. It requires Claude Code v2.1.174 or later. On an `async_launched` response, `resolvedModel` names the model in use when the agent moved to the background, so a swap that happened before backgrounding is reflected there. `modelsUsed` and the backgrounding-time `resolvedModel` behavior require Claude Code v2.1.212 or later.

AskUserQuestion

Asks the user one to four multiple-choice questions.

Field	Type	Example	Description
<code>questions</code>	array	<code>[{"question": "Which framework?", "header": "Framework", "options": [{"label": "React"}], "multiSelect": false}]</code>	Questions to present, each with a <code>question</code> string, short <code>header</code> , <code>options</code> array, and optional <code>multiSelect</code> flag
<code>answers</code>	object	<code>{"Which framework?": "React"}</code>	Optional. Maps question text to the selected option label. Multi-select answers join labels with commas. Claude doesn't set this field; supply it via <code>updatedInput</code> to answer programmatically

Field	Type	Example	Description
<code>plan</code>	string	<code>""" Refactor auth\n1. Extract..."""</code>	Plan content in Markdown. Injected from the plan file on disk
<code>planFilePath</code>	string	<code>"/Users/.../plans/re factor-auth.md"</code>	Path to the plan file. Injected
<code>allowedPrompts</code>	array	<code>[{"tool": "Bash", "prompt": "run tests"}]</code>	Deprecated. Claude Code accepts the field but ignores it. Before v2.1.205, it carried prompt-based permissions Claude requested to implement the plan

In `PostToolUse`, `tool_response` is an object with `plan` and `filePath` fields holding the approved plan, plus internal status flags. Read `tool_response.plan` for the plan content rather than re-reading the file from disk.

PreToolUse decision control

`PreToolUse` hooks can control whether a tool call proceeds. Unlike other hooks that use a top-level `decision` field, `PreToolUse` returns its decision inside a `hookSpecificOutput` object. This gives it richer control: four outcomes (allow, deny, ask, or defer) plus the ability to modify tool input before execution.

gracefully so the tool can be resumed later. Deny and ask rules are still evaluated regardless of what the hook returns

<code>permissionDecisionReason</code>	For <code>"allow"</code> and <code>"ask"</code> , shown to the user but not Claude. For <code>"deny"</code> , shown to Claude. For <code>"defer"</code> , ignored
<code>updatedInput</code>	Modifies the tool's input parameters before execution. Replaces the entire input object, so include unchanged fields alongside modified ones. Combine with <code>"allow"</code> to auto-approve, or <code>"ask"</code> to show the modified input to the user. For <code>"defer"</code> , ignored
<code>additionalContext</code>	String added to Claude's context alongside the tool result. Ignored when <code>permissionDecision</code> is <code>"defer"</code> . See Add context for Claude

When multiple PreToolUse hooks return different decisions, precedence is `deny` > `defer` > `ask` > `allow`.

A hook that blocks by exiting 2 routes the same way as `"deny"`: Claude sees the stderr message as the denial reason.

When a hook returns `"ask"`, the permission prompt displayed to the user includes a label identifying where the hook came from: `[settings]` for a hook from any settings file or from agent frontmatter, `[plugin:<name>]` for a plugin's hook, or `[skill]` for a hook from skill frontmatter. This helps users understand which configuration source is requesting confirmation.

A hook's `"ask"` also forces a permission prompt in auto mode: the classifier can still deny the tool call, but it can't approve the call silently. Before v2.1.211, the classifier could approve a Bash command running outside the sandbox without showing the prompt the hook requested; the classifier still applied its own safety rules to that command, and a hook `"deny"` was always honored.

```
"permissionDecisionReason": "My reason here",
"updatedInput": {
  "field_to_modify": "new value"
},
"additionalContext": "Current environment: production. Proceed with caution."
}
}
```

`AskUserQuestion` and `ExitPlanMode` require user interaction and normally block in **non-interactive mode** with the `-p` flag. Returning `permissionDecision: "allow"` together with `updatedInput` satisfies that requirement: the hook reads the tool's input from stdin, collects the answer through your own UI, and returns it in `updatedInput` so the tool runs without prompting. Returning `"allow"` alone is not sufficient for these tools. For `AskUserQuestion`, echo back the original `questions` array and add an `answers` object mapping each question's text to the chosen answer.

As of v2.1.199, an MCP tool whose server marks it with

`_meta["anthropic/requiresUserInteraction"]` is stricter: a hook can't skip its approval prompt with `"allow"`, with or without `updatedInput`, because Claude Code can't confirm the hook collected the interaction the tool needs.

- ❗ `PreToolUse` previously used top-level `decision` and `reason` fields, but these are deprecated for this event. Use `hookSpecificOutput.permissionDecision` and `hookSpecificOutput.permissionDecisionReason` instead. The deprecated values `"approve"` and `"block"` map to `"allow"` and `"deny"` respectively. Other events like `PostToolUse` and `Stop` continue to use top-level `decision` and `reason` as their current format.

Defer a tool call for later

`"defer"` is for integrations that run `claude -p` as a subprocess and read its JSON output, such as an Agent SDK app or a custom UI built on top of Claude Code. It lets that calling process pause Claude at a tool call, collect input through its own interface, and resume where it left off. Claude Code honors this value only in **non-interactive mode** with the `-p` flag. In interactive sessions it logs a warning and ignores the hook result.

The `AskUserQuestion` tool is the typical case: Claude wants to ask the user something, but there is no terminal to answer in. The round trip works like this:

its own UI, and waits for an answer.

4. The calling process runs `claude -p --resume <session-id>`. The same tool call fires `PreToolUse` again.
5. The hook returns `permissionDecision: "allow"` with the answer in `updatedInput`. The tool executes and Claude continues.

The `deferred_tool_use` field carries the tool's `id`, `name`, and `input`. The `input` is the parameters Claude generated for the tool call, captured before execution:

```
{
  "type": "result",
  "subtype": "success",
  "stop_reason": "tool_deferred",
  "session_id": "abc123",
  "deferred_tool_use": {
    "id": "toolu_01abc",
    "name": "AskUserQuestion",
    "input": { "questions": [{ "question": "Which framework?", "header": "Framework",
  }
}
```

There is no timeout or retry limit. The session remains on disk until you resume it, subject to the `cleanupPeriodDays` retention sweep, which deletes session files after 30 days by default, following the [retention sweep rules](#). If the answer is not ready when you resume, the hook can return `"defer"` again and the process exits the same way. The calling process controls when to break the loop by eventually returning `"allow"` or `"deny"` from the hook.

`"defer"` only works when Claude makes a single tool call in the turn. If Claude makes several tool calls at once, `"defer"` is ignored with a warning and the tool proceeds through the normal permission flow. The constraint exists because resume can only re-run one tool: there is no way to defer one call from a batch without leaving the others unresolved.

If the deferred tool is no longer available when you resume, the process exits with `stop_reason: "tool_deferred_unavailable"` and `is_error: true` before the hook fires. This happens when an MCP server that provided the tool is not connected for the resumed session. The `deferred_tool_use` payload is still included so you can identify which tool went missing

the run in the permission mode a new `claude -p run` would start in, so pass `--permission-mode` or `--dangerously-skip-permissions` again if the deferred session used one. When you resume with `claude --resume <session-id>` without `-p`, Claude Code restores the stored permission mode, with the exceptions listed in [permission mode on resume](#).

PermissionRequest

Runs when Claude Code is about to ask you for permission to use a tool. In sessions that can't show a prompt, such as background subagents in [non-interactive mode](#), Claude Code still runs these hooks, and if no hook returns a decision, it denies the tool call. Use [PermissionRequest decision control](#) to allow or deny on behalf of the user.

Use this event when you need a signal the moment Claude asks for permission to use a tool. Claude Code runs a [Notification](#) hook with the `permission_prompt` type only after the prompt has waited about six seconds.

Claude Code doesn't run PermissionRequest hooks for a sandboxed command's [network request](#). To get a signal for that prompt, use the `permission_prompt` notification type.

Matches on tool name, same values as PreToolUse.

PermissionRequest input

PermissionRequest hooks receive `tool_name` and `tool_input` fields like PreToolUse hooks, but without `tool_use_id`. An optional `permission_suggestions` array contains the "always allow" options the user would normally see in the permission dialog.

PreToolUse hooks run before every tool call, whether or not it needs permission. PermissionRequest hooks run only when Claude Code is about to ask you for permission, or when it would otherwise auto-deny a call that can't prompt. Neither event fires for [EndConversation](#).

```

"permission_mode": "default",
"hook_event_name": "PermissionRequest",
"tool_name": "Bash",
"tool_input": {
  "command": "rm -rf node_modules",
  "description": "Remove node_modules directory"
},
"permission_suggestions": [
  {
    "type": "addRules",
    "rules": [{ "toolName": "Bash", "ruleContent": "rm -rf node_modules" }],
    "behavior": "allow",
    "destination": "localSettings"
  }
]
}

```

PermissionRequest decision control

`PermissionRequest` hooks can allow or deny permission requests. In addition to the **JSON output fields** available to all hooks, your hook script can return a `decision` object with these event-specific fields:

Field	Description
<code>behavior</code>	"allow" grants the permission, "deny" denies it. <u>Deny and ask rules</u> are still evaluated, so a hook returning "allow" doesn't override a matching deny rule
<code>updatedInput</code>	For "allow" only: modifies the tool's input parameters before execution. Replaces the entire input object, so include unchanged fields alongside modified ones. The modified input is re-evaluated against deny and ask rules
<code>updatedPermissions</code>	For "allow" only: array of <u>permission update entries</u> to apply, such as adding an allow rule or changing the session permission mode
<code>message</code>	For "deny" only: tells Claude why the permission was denied
<code>interrupt</code>	For "deny" only: if <code>true</code> , stops Claude

A hook that exits 2 without a `decision` object leaves the permission flow unchanged, and its stderr is discarded. Only the `decision` object can grant or deny the request.

```

    "behavior": "allow",
    "updatedInput": {
      "command": "npm run lint"
    }
  }
}
}

```

Permission update entries

The `updatedPermissions` output field and the `permission_suggestions` **input field** both use the same array of entry objects. Each entry has a `type` that determines its other fields, and a `destination` that controls where the change is written.

type	Fields	Effect
<code>addRules</code>	<code>rules</code> , <code>behavior</code> , <code>destination</code>	Adds permission rules. <code>rules</code> is an array of <code>{toolName, ruleContent?}</code> objects. Omit <code>ruleContent</code> to match the whole tool. <code>behavior</code> is <code>"allow"</code> , <code>"deny"</code> , or <code>"ask"</code>
<code>replaceRules</code>	<code>rules</code> , <code>behavior</code> , <code>destination</code>	Replaces all rules of the given <code>behavior</code> at the <code>destination</code> with the provided <code>rules</code>
<code>removeRules</code>	<code>rules</code> , <code>behavior</code> , <code>destination</code>	Removes matching rules of the given <code>behavior</code>
<code>setMode</code>	<code>mode</code> , <code>destination</code>	Changes the permission mode. Valid modes are <code>default</code> , <code>auto</code> , <code>acceptEdits</code> , <code>dontAsk</code> , <code>bypassPermissions</code> , <code>plan</code> , and <code>manual</code> as an alias for <code>default</code> . The <code>manual</code> alias requires Claude Code v2.1.200 or later
<code>addDirectories</code>	<code>directories</code> , <code>destination</code>	Adds working directories. <code>directories</code> is an array of path strings
<code>removeDirectories</code>	<code>directories</code> , <code>destination</code>	Removes working directories

❗ `setMode` with `bypassPermissions` only takes effect if the session was launched with bypass mode already available: `--dangerously-skip-permissions` , `--permission-mode bypassPermissions` , `--allow-dangerously-skip-permissions` , or `permissions.defaultMode: "bypassPermissions"` in settings, and the mode is not disabled by `permissions.disableBypassPermissionsMode` or by starting the session in **restricted mode**. Otherwise the update is a no-op. `bypassPermissions` is never persisted as `defaultMode` regardless of `destination` .

<code>session</code>	in-memory only, discarded when the session ends
<code>localSettings</code>	<code>.claude/settings.local.json</code>
<code>projectSettings</code>	<code>.claude/settings.json</code>
<code>userSettings</code>	<code>~/.claude/settings.json</code>

A hook can echo one of the `permission_suggestions` it received as its own `updatedPermissions` output, which is equivalent to the user selecting that “always allow” option in the dialog.

PostToolUse

Runs immediately after a tool completes successfully.

Matches on tool name, same values as PreToolUse.

Match more broadly when the tool name isn't the right filter:

- To run a hook after any tool completes successfully, omit the `matcher` or set it to `"*"`. Your hook can then discover what changed itself, for example by running `git status --porcelain`, which also lists untracked files that `git diff` misses. For tool calls that fail, add the same hook under **PostToolUseFailure**.
- To run a hook when a specific file changes on disk, whatever wrote it, use **FileChanged**. Claude Code doesn't run a `PostToolUse` hook matching `Edit|Write` when a `Bash` command or a process outside Claude Code rewrites the same file.

PostToolUse input

`PostToolUse` hooks fire after a tool has already executed successfully. The input includes both `tool_input`, the arguments sent to the tool, and `tool_response`, the result it returned. The exact schema for both depends on the tool. File-tool `tool_input` paths arrive in the same format as for **PreToolUse**: always absolute, with the platform's native separators, so backslashes on Windows.

```
"permission_mode": "default",
"hook_event_name": "PostToolUse",
"tool_name": "Write",
"tool_input": {
  "file_path": "/path/to/file.txt",
  "content": "file content"
},
"tool_response": {
  "filePath": "/path/to/file.txt",
  "success": true
},
"tool_use_id": "toolu_01ABC123...",
"duration_ms": 12
}
```

Field	Description
<code>duration_ms</code>	Optional. Tool execution time in milliseconds. Excludes time spent in permission prompts and PreToolUse hooks

PostToolUse decision control

`PostToolUse` hooks can provide feedback to Claude after tool execution. In addition to the [JSON output fields](#) available to all hooks, your hook script can return these event-specific fields:

Field	Description
<code>decision</code>	<code>"block"</code> adds the <code>reason</code> next to the tool result. Claude still sees the original output; to replace it, use <code>updatedToolOutput</code>
<code>reason</code>	Explanation shown to Claude when <code>decision</code> is <code>"block"</code>
<code>additionalContext</code>	String added to Claude's context alongside the tool result. See Add context for Claude
<code>classifierContext</code>	Short note about this call's result for the auto mode classifier rather than for Claude. See Annotate a result for the auto mode classifier . Requires Claude Code v2.1.236 or later
<code>updatedToolOutput</code>	Replaces the tool's output with the provided value before it is sent to Claude. The value must match the tool's output shape
<code>updatedMCPToolOutput</code>	Replaces the output for MCP tools only. Prefer <code>updatedToolOutput</code> , which works for all tools

```
"hookSpecificOutput": {
  "hookEventName": "PostToolUse",
  "additionalContext": "Additional information for Claude",
  "updatedToolOutput": {
    "stdout": "[redacted]",
    "stderr": "",
    "interrupted": false,
    "isImage": false
  }
}
```

⚠ `updatedToolOutput` only changes what Claude sees. The tool has already run by the time the hook fires, so any files written, commands executed, or network requests sent have already taken effect. Telemetry such as OpenTelemetry tool spans and analytics events also captures the original output before the hook runs. To prevent or modify a tool call before it runs, use a **`PreToolUse`** hook instead.

The replacement value must match the tool's output shape. Built-in tools return structured objects rather than plain strings. For example, `Bash` returns an object with `stdout`, `stderr`, `interrupted`, and `isImage` fields. For built-in tools, a value that doesn't match the tool's output schema is ignored and the original output is used. MCP tool output is passed through without schema validation. Stripping error details that Claude needs can cause it to proceed on a false assumption.

Annotate a result for the auto mode classifier

Return `classifierContext` to send a short note about the tool call's result to the **auto mode** classifier rather than to Claude. The classifier **never receives tool results themselves**, so this field is the supported way to tell it something about what a call returned before it reviews later actions. The field requires Claude Code v2.1.236 or later.

The example below tells the classifier where a query's output came from:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PostToolUse",
    "classifierContext": "This query ran against the staging database, not production."
  }
}
```

never establishes user intent, and if it claims you approved or requested something, the classifier checks that claim against your own messages in the conversation

- **In-process Agent SDK callbacks:** when an application embedding Claude Code registers the hook as a **TypeScript SDK callback** and returns the note during the live session, the classifier may weigh a user statement relayed in the note as user intent. Such a statement can satisfy a consent requirement the classifier would accept from a message you send, but it never lifts a block that your own message couldn't lift either. After a session resumes, Claude Code treats restored notes as unverified context. When hooks from both groups annotate the same call, the classifier treats the combined note as unverified

Claude Code applies these limits when delivering the note:

- **Length:** Claude Code caps the notes for one tool call at 2,000 characters and truncates the rest. The cap is shared across every hook that responds to that call
- **Synchronous responses only:** Claude Code ignores the field in the response of a hook that **runs in the background**, because that response arrives after Claude Code records the tool result
- **Calls that the classifier doesn't record:** the classifier's transcript omits read-only lookups such as file reads and searches. Claude Code discards a note attached to one of those calls
- **Interaction with rewrites:** when the note describes output you're replacing with `updatedToolOutput`, return both fields in the same hook response. Claude Code drops the note if that rewrite is rejected or another hook's rewrite replaces it. Claude Code delivers a note you return without a rewrite even when another hook rewrites the output

⚠ The classifier reads content you place in `classifierContext` as information from the application hosting the session, so don't copy untrusted tool output or third-party text into it. Keep the note to a short assertion about this one call, such as a fact about its origin or a user statement about it; don't use the field to deliver unrelated messages or a stream of events.

PostToolUseFailure

Runs when a tool that started executing fails: the tool threw an error, or an MCP tool returned an error result. Use this to log failures, send alerts, or provide corrective feedback to Claude.

Matches on tool name, same values as PreToolUse.

PostToolUseFailure input

PostToolUseFailure hooks receive the same `tool_name` and `tool_input` fields as PostToolUse, along with error information as top-level fields. For example, a failed `npm test` command might deliver:

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9k",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "PostToolUseFailure",
  "tool_name": "Bash",
  "tool_input": {
    "command": "npm test",
    "description": "Run test suite"
  },
  "tool_use_id": "toolu_01ABC123...",
  "error": "Exit code 1\nError: Cannot find module 'express'",
  "is_interrupt": false,
  "duration_ms": 4187
}
```

Field	Description
<code>error</code>	String describing what went wrong. The format depends on the tool that failed
<code>is_interrupt</code>	Optional boolean. True when the failure reached Claude Code as an abort rather than as an error the tool reported. Cancelling a running tool does not fire this hook; the tool result carries the interruption message instead
<code>duration_ms</code>	Optional. Tool execution time in milliseconds. Excludes time spent in permission prompts and PreToolUse hooks

The `error` string is generally the same text Claude receives as the failed tool's result. Its format varies by tool and failure. Key your hook on `tool_name`, `is_interrupt`, and the `Exit code N` first line; treat the rest of the string as display text, not a stable format.

- Claude Code middle-truncates strings longer than 10,000 characters around a `... [N characters truncated] ...` marker, and can insert lines of its own, such as `Command timed out after 2m 0s`

PostToolUseFailure decision control

`PostToolUseFailure` hooks can provide context to Claude after a tool failure. In addition to the **JSON output fields** available to all hooks, your hook script can return these event-specific fields:

Field	Description
<code>additionalContext</code>	String added to Claude's context alongside the error. See Add context for Claude

```
{
  "hookSpecificOutput": {
    "hookEventName": "PostToolUseFailure",
    "additionalContext": "Additional information about the failure for Claude"
  }
}
```

PostToolBatch

Runs once after every tool call in a batch has resolved, before Claude Code sends the next request to the model. `PostToolUse` fires once per tool, which means it fires concurrently when Claude makes parallel tool calls. `PostToolBatch` fires exactly once with the full batch, so it is the right place to inject context that depends on the set of tools that ran rather than on any single tool. There is no matcher for this event.

PostToolBatch input

In addition to the **common input fields**, `PostToolBatch` hooks receive `tool_calls`, an array describing every tool call in the batch:

```

"permission_mode": "default",
"hook_event_name": "PostToolBatch",
"tool_calls": [
  {
    "tool_name": "Read",
    "tool_input": {"file_path": "../ledger/accounts.py"},
    "tool_use_id": "toolu_01...",
    "tool_response": "      1\tfrom __future__ import annotations\n      2\t..."
  },
  {
    "tool_name": "Read",
    "tool_input": {"file_path": "../ledger/transactions.py"},
    "tool_use_id": "toolu_02...",
    "tool_response": "      1\tfrom __future__ import annotations\n      2\t..."
  }
]
}

```

`tool_response` contains the same content the model receives in the corresponding `tool_result` block. The value is a serialized string or content-block array, exactly as the tool emitted it. For `Read`, that means line-number-prefixed text rather than raw file contents. Responses can be large, so parse only the fields you need.

❗ The `tool_response` shape differs from `PostToolUse`'s. `PostToolUse` passes the tool's structured `Output` object, such as `{filePath: "...", success: true}` for `Write`; `PostToolBatch` passes the serialized `tool_result` content the model sees.

PostToolBatch decision control

`PostToolBatch` hooks can inject context for Claude. In addition to the **JSON output fields** available to all hooks, your hook script can return these event-specific fields:

Field	Description
<code>additionalContext</code>	Context string injected once before the next model call. See <u>Add context for Claude</u> for delivery details, what to put in it, and how resumed sessions handle past values

```
}  
}
```

Returning `decision: "block"` or `continue: false` stops the agentic loop before the next model call. The blocking message comes from the JSON `reason` or `stopReason`, or from `stderr` on exit 2. You see it as a warning in the transcript, and it stays in the conversation, so Claude sees it when the conversation continues.

PermissionDenied

Runs when **auto mode** denies a tool call, including when it denies without a classifier verdict because **a safety check separate from auto mode refused the classifier's own request** or its response didn't parse. This hook only fires in auto mode: it doesn't run when you manually deny a permission dialog, when a `PreToolUse` hook blocks a call, or when a `deny` rule matches. Use it to log denials, adjust configuration, or tell the model it may retry the tool call.

Matches on tool name, same values as `PreToolUse`.

PermissionDenied input

In addition to the **common input fields**, `PermissionDenied` hooks receive `tool_name`, `tool_input`, `tool_use_id`, and `reason`.

```
{  
  "session_id": "abc123",  
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9k",  
  "cwd": "/Users/...",  
  "permission_mode": "auto",  
  "hook_event_name": "PermissionDenied",  
  "tool_name": "Bash",  
  "tool_input": {  
    "command": "rm -rf /tmp/build",  
    "description": "Clean build directory"  
  },  
  "tool_use_id": "toolu_01ABC123...",  
  "reason": "Blocked by classifier"  
}
```

its response didn't parse, the reason starts with `Auto mode could not evaluate this action and is blocking it for safety`. For a denial because the classifier model was unavailable, the reason is the fixed text `Classifier unavailable`. See [Review denials](#)

PermissionDenied decision control

PermissionDenied hooks can tell the model it may retry the denied tool call. Return a JSON object with `hookSpecificOutput.retry` set to `true`:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PermissionDenied",
    "retry": true
  }
}
```

When `retry` is `true`, Claude Code adds a message to the conversation telling the model it may retry the tool call. Claude Code doesn't reverse the denial itself. If your hook doesn't return JSON, or returns `retry: false`, the denial stands and the model receives the original rejection message.

Claude Code ignores `retry: true` when the classifier produced **no verdict on the action**: its response didn't parse, or a safety check separate from auto mode refused the classifier's own request. For those denials, Claude Code already tells the model in the rejection message whether to retry later or move on.

Notification

Runs when Claude Code sends notifications. Matches on notification type. Omit the matcher to run hooks for all notification types.

You receive these hook events even with desktop notifications turned off: the `preferredNotifChannel` setting, including `notifications_disabled`, changes only how you're alerted, not whether your hook runs.

<code>idle_prompt</code>	Claude finished responding about 60 seconds ago and you haven't typed since
<code>auth_success</code>	Authentication completes
<code>elicitation_dialog</code>	An MCP server opens an elicitation form and you haven't typed for about six seconds
<code>elicitation_url_dialog</code>	An MCP server asks you to open a browser URL and you haven't typed for about six seconds
<code>elicitation_complete</code>	An MCP server reports that a <u>URL-mode elicitation</u> is complete
<code>elicitation_response</code>	An MCP elicitation response is sent back to the server
<code>agent_needs_input</code>	A background session starts waiting on your input. Fires only while <u>agent view</u> is open in a terminal
<code>agent_completed</code>	A background session finishes or fails. Fires only while <u>agent view</u> is open in a terminal
<code>quota_auto_resume_fired</code>	Claude Code continues your task after a claude.ai usage limit paused it: at the reset, or sooner when something you do in Claude Code during the wait, such as adding usage credits, upgrading your plan, or switching models, makes usage available again, with the <u>model-setting exception</u>
<code>quota_auto_resume_stale</code>	A claude.ai usage limit reset while your computer slept for more than about 30 minutes. Claude Code waits for you to press <code>Enter</code> instead of continuing. After a shorter sleep it continues and fires <code>quota_auto_resume_fired</code> instead
<code>quota_auto_resume_disabled</code>	Claude Code ends its wait for a claude.ai usage limit without continuing your task: <u>autoContinueAtUsageLimit</u> turned off or the reset moved more than 24 hours away during a wait Claude Code started on its own, the continued task kept hitting the limit, or the continuation was blocked before it reached the model. Doesn't fire when you press <code>Esc</code> or <code>Ctrl+C</code> , or pick Don't continue automatically

The `agent_needs_input` and `agent_completed` types require Claude Code v2.1.198 or later.

The `quota_auto_resume_fired`, `quota_auto_resume_stale`, and `quota_auto_resume_disabled` types require Claude Code v2.1.234 or later.

In terminal sessions, `permission_prompt` for a sandboxed command's network request requires Claude Code v2.1.246 or later.

the permission prompt appears, and each keystroke defers it. To run a hook immediately when Claude asks for permission to use a tool, use [PermissionRequest](#) instead.

- Expect `idle_prompt` about 60 seconds after Claude finishes responding, and only if you haven't typed since. Claude Code doesn't send `idle_prompt` while it waits for a claude.ai usage limit to reset. When the wait ends on its own, one of the `quota_auto_resume_*` types fires instead.
- Expect `elicitation_dialog` for an elicitation form, or `elicitation_url_dialog` for a browser URL request, once you haven't typed for about six seconds. Both share the same six-second gate as `permission_prompt`: the timer starts when the dialog appears, and each keystroke defers it.

Claude Code times `permission_prompt` differently in sessions where it sends permission requests to the Agent SDK's `canUseTool` **callback**, which is how Claude Desktop and the VS Code extension host Claude Code:

- Expect `permission_prompt` about six seconds after Claude asks for permission. Claude Code doesn't defer it while you type.
- If you or a [PermissionRequest](#) hook answer sooner, Claude Code doesn't run `permission_prompt`.
- Set `CLAUDE_CODE_DISABLE_PERMISSION_PROMPT_NOTIFY_HOOKS` to `1` to turn `permission_prompt` off in these sessions.

Before v2.1.233, `permission_prompt` didn't fire in these sessions.

Use separate matchers to run different handlers depending on the notification type. This configuration triggers a permission-specific alert script when Claude needs permission approval and a different notification when Claude has been idle:

```
    "matcher": "permission_prompt",
    "hooks": [
      {
        "type": "command",
        "command": "/path/to/permission-alert.sh"
      }
    ]
  },
  {
    "matcher": "idle_prompt",
    "hooks": [
      {
        "type": "command",
        "command": "/path/to/idle-notification.sh"
      }
    ]
  }
]
```

Notification input

In addition to the **common input fields**, Notification hooks receive `message` with the notification text, an optional `title`, and `notification_type` indicating which type fired.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9b...",
  "cwd": "/Users/...",
  "hook_event_name": "Notification",
  "message": "Claude needs your permission",
  "title": "Permission needed",
  "notification_type": "permission_prompt"
}
```

Notification hooks can't block or modify notifications. Claude Code discards their `systemMessage` and `continue` fields but still emits `terminalSequence`, which is what the desktop notification

Runs when a Claude Code subagent is spawned via the Agent tool. Supports matchers to filter by agent type name. For built-in agents, this is the agent name like `general-purpose`, `Explore`, or `Plan`. For **custom subagents**, this is the `name` field from the agent's frontmatter, not the filename.

For subagents shipped by a **plugin**, the agent type is the plugin-scoped identifier such as `my-plugin:reviewer`, not the bare frontmatter name. The colon places a plugin-scoped name on the regular-expression path, so anchor the matcher with `^` and `$` for an exact match: `^my-plugin:reviewer$`.

SubagentStart input

In addition to the **common input fields**, SubagentStart hooks receive `agent_id` with the unique identifier for the subagent and `agent_type` with the agent name that the matcher filters on.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9t",
  "cwd": "/Users/...",
  "hook_event_name": "SubagentStart",
  "agent_id": "agent-abc123",
  "agent_type": "Explore"
}
```

SubagentStart hooks can't block subagent creation, but they can inject context into the subagent. In addition to the **JSON output fields** available to all hooks, you can return:

Field	Description
<code>additionalContext</code>	String added to the subagent's context at the start of its conversation, before its first prompt. See Add context for Claude

```
}  
}
```

SubagentStop

Runs when a Claude Code subagent has finished responding. Matches on agent type, same values as SubagentStart.

SubagentStop input

In addition to the **common input fields**, SubagentStop hooks receive `stop_hook_active`, `agent_id`, `agent_type`, `agent_transcript_path`, and `last_assistant_message`. The `agent_type` field is the value used for matcher filtering. The `transcript_path` is the main session's transcript, while `agent_transcript_path` is the subagent's own transcript stored in a nested `subagents/` folder. The `last_assistant_message` field contains the text content of the subagent's final response, so hooks can access it without parsing the transcript file.

SubagentStop hooks also receive the `background_tasks` and `session_crons` arrays described under **Stop input**. Both arrays are scoped to the parent session, not the subagent.

```
{  
  "session_id": "abc123",  
  "transcript_path": "~/.claude/projects/.../abc123.jsonl",  
  "cwd": "/Users/...",  
  "permission_mode": "default",  
  "hook_event_name": "SubagentStop",  
  "stop_hook_active": false,  
  "agent_id": "def456",  
  "agent_type": "Explore",  
  "agent_transcript_path": "~/.claude/projects/.../abc123/subagents/agent-def456.jsonl",  
  "last_assistant_message": "Analysis complete. Found 3 potential issues...",  
  "background_tasks": [],  
  "session_crons": []  
}
```

SubagentStop hooks use the same decision control format as **Stop hooks**, including

`hookSpecificOutput.additionalContext` with `hookEventName` set to `"SubagentStop"` for non-error feedback that keeps the subagent running. Returning `decision: "block"` with a `reason`

TaskCreated

Runs when a task is being created via the `TaskCreate` tool. Use this to enforce naming conventions, require task descriptions, or prevent certain tasks from being created. In a **session without the Task tools**, this event doesn't fire.

TaskCreated hooks don't support matchers and fire on every occurrence.

TaskCreated input

In addition to the **common input fields**, TaskCreated hooks receive `task_id` , `task_subject` , and optionally `task_description` , `teammate_name` , and `team_name` .

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9b...",
  "cwd": "/Users/...",
  "hook_event_name": "TaskCreated",
  "task_id": "task-001",
  "task_subject": "Implement user authentication",
  "task_description": "Add login and signup endpoints",
  "teammate_name": "implementer",
  "team_name": "session-a1b2c3d4"
}
```

Field	Description
<code>task_id</code>	Identifier of the task being created
<code>task_subject</code>	Title of the task
<code>task_description</code>	Detailed description of the task. May be absent
<code>teammate_name</code>	Name of the teammate creating the task. May be absent
<code>team_name</code>	Deprecated. Session-derived team name; will be removed in a future release

- **Exit code 2:** Claude Code returns the stderr text as the message.
- **JSON** `{"decision": "block", "reason": "..."} :` Claude Code returns `reason` as the message.

This example blocks tasks whose subjects don't follow the required format:

```
#!/bin/bash
INPUT=$(cat)
TASK_SUBJECT=$(echo "$INPUT" | jq -r '.task_subject')

if [[ ! "$TASK_SUBJECT" =~ ^\[TICKET-[0-9]+\] ]]; then
    echo "Task subject must start with a ticket number, e.g. '[TICKET-123] Add feature'"
    exit 2
fi

exit 0
```

TaskCompleted

Runs when a task is being marked as completed. This fires in two situations: when any agent explicitly marks a task as completed through the TaskUpdate tool, or when an agent team teammate finishes its turn with in-progress tasks. Use this to enforce completion criteria like passing tests or lint checks before a task can close.

TaskCompleted hooks don't support matchers and fire on every occurrence.

TaskCompleted input

In addition to the common input fields, TaskCompleted hooks receive `task_id` , `task_subject` , and optionally `task_description` , `teammate_name` , and `team_name` .

```
"permission_mode": "default",
"hook_event_name": "TaskCompleted",
"task_id": "task-001",
"task_subject": "Implement user authentication",
"task_description": "Add login and signup endpoints",
"teammate_name": "implementer",
"team_name": "session-a1b2c3d4"
}
```

Field	Description
<code>task_id</code>	Identifier of the task being completed
<code>task_subject</code>	Title of the task
<code>task_description</code>	Detailed description of the task. May be absent
<code>teammate_name</code>	Name of the teammate completing the task. May be absent
<code>team_name</code>	Deprecated. Session-derived team name; will be removed in a future release

TaskCompleted decision control

TaskCompleted hooks support two ways to control task completion:

- **Exit code 2:** the task is not marked as completed and the stderr message is fed back to the model as feedback.
- **JSON** `{"continue": false, "stopReason": "..."} : when a teammate finishing its turn triggered the event, stops the teammate entirely, matching Stop hook behavior. The stopReason is shown to the user. When the TaskUpdate tool triggered the event, Claude Code ignores continue: false; exit code 2 still blocks the completion.`

This example runs tests and blocks task completion if they fail:

```
# Run the test suite
if ! npm test 2>&1; then
  echo "Tests not passing. Fix failing tests before completing: $TASK_SUBJECT" >&2
  exit 2
fi

exit 0
```

Stop

Runs when the main Claude Code agent has finished responding. Does not run if the stoppage occurred due to a user interrupt. API errors fire **StopFailure** instead.

 The `/goal` command is a built-in shortcut for a session-scoped prompt-based Stop hook. Use it when you want Claude to keep working toward a condition without writing hook configuration.

Stop input

In addition to the **common input fields**, Stop hooks receive `stop_hook_active`, `last_assistant_message`, `background_tasks`, and `session_crons`. The `stop_hook_active` field is `true` when Claude Code is already continuing as a result of a stop hook. Check this value or process the transcript to avoid blocking on a condition that will never resolve. Claude Code overrides the hook and ends the turn after 8 consecutive blocks.

The `last_assistant_message` field contains the text content of Claude's final response, so hooks can access it without parsing the transcript file. For hooks that act on the just-completed turn, such as read-aloud or notification hooks, use this field rather than reading `transcript_path`: the transcript file isn't guaranteed to include the final message at Stop time on all versions.

The `background_tasks` and `session_crons` arrays let hooks distinguish “session is done” from “session is paused waiting for background work to wake it back up”. Both arrays are present when the task registry is reachable and are empty when nothing is in flight or scheduled.

Each entry in `background_tasks` describes one in-flight task and uses these fields:

Field	Description
<code>id</code>	Task identifier

status	Current task status
description	Free-text description, capped at 1000 characters with an in-string ... [+N chars] marker when clipped
command	Shell command line, capped at 1000 characters. Present only for shell tasks
agent_type	Subagent type name. Present only for subagent tasks
server	MCP server name. Present only for monitor and MCP task tasks
tool	MCP tool name. Present only for monitor and MCP task tasks
name	Workflow name. Present only for workflow tasks

Each entry in session_crons describes one session-scoped scheduled wakeup, sourced from CronCreate , ScheduleWakeup , and /loop :

Field	Description
id	Cron task identifier
schedule	Cron expression, for example 0 9 * * 1-5
recurring	false for one-shot wakeups whose schedule encodes a single fire time, true for tasks that re-fire on every match
prompt	Prompt submitted when the cron fires, capped at 1000 characters with the same ... [+N chars] marker

This example shows a Stop input with one in-flight shell task and one recurring cron:

```

"permission_mode": "default",
"hook_event_name": "Stop",
"stop_hook_active": true,
"last_assistant_message": "I've completed the refactoring. Here's a summary...",
"background_tasks": [
  {
    "id": "task-001",
    "type": "shell",
    "status": "running",
    "description": "tail logs",
    "command": "tail -f /var/log/syslog"
  }
],
"session_crons": [
  {
    "id": "cron-001",
    "schedule": "0 9 * * 1-5",
    "recurring": true,
    "prompt": "check the build"
  }
]
}

```

Stop decision control

`Stop` and `SubagentStop` hooks can control whether Claude continues. In addition to the [JSON output fields](#) available to all hooks, your hook script can return these event-specific fields:

Field	Description
<code>decision</code>	<code>"block"</code> prevents Claude from stopping. Omit to allow Claude to stop
<code>reason</code>	Required when <code>decision</code> is <code>"block"</code> . Tells Claude why it should continue
<code>hookSpecificOutput.additionalContext</code>	Non-error feedback for Claude. The conversation continues so Claude can act on it, but unlike <code>decision: "block"</code> it is shown in the transcript as hook feedback rather than a hook error

A hook that blocks by exiting 2 routes the same way as `reason`: Claude receives the stderr message as the explanation for why it should continue.

Use `additionalContext` when the hook is working as designed and giving Claude guidance, such as “run the test suite before finishing”. It keeps the conversation going through the same loop protections as `decision: "block"`, namely the `stop_hook_active` input and the 8-consecutive-continuation cap, but the transcript labels it `Stop hook feedback` and no hook error notification is shown:

```
{
  "hookSpecificOutput": {
    "hookEventName": "Stop",
    "additionalContext": "Please run the test suite before finishing"
  }
}
```

StopFailure

Runs instead of **Stop** when the turn ends due to an API error. Claude Code ignores the hook's output and exit code, apart from `terminalSequence`. Use this to log failures, send alerts, or take recovery actions when Claude can't complete a response due to rate limits, authentication problems, or other API errors.

StopFailure input

In addition to the **common input fields**, StopFailure hooks receive `error`, optional `error_details`, and optional `last_assistant_message`. The `error` field identifies the error type and is used for matcher filtering.

Field	Description
<code>error</code>	Error type: <code>rate_limit</code> , <code>overloaded</code> , <code>authentication_failed</code> , <code>oauth_org_not_allowed</code> , <code>billing_error</code> , <code>invalid_request</code> , <code>model_not_found</code> , <code>server_error</code> , <code>max_output_tokens</code> , or <code>unknown</code>
<code>error_details</code>	Additional details about the error, when available
<code>last_assistant_message</code>	The rendered error text shown in the conversation. Unlike <code>Stop</code> and <code>SubagentStop</code> , where this field holds Claude's conversational output, for <code>StopFailure</code> it contains the API error string itself, such as <code>"API Error: Rate limit reached"</code>

```
"hook_event_name": "StopFailure",
"error": "rate_limit",
"error_details": "429 Too Many Requests",
"last_assistant_message": "API Error: Rate limit reached"
}
```

StopFailure hooks have no decision control. They run for notification and logging purposes only.

TeammateIdle

Runs when an **agent team** teammate is about to go idle after finishing its turn. Use this to enforce quality gates before a teammate stops working, such as requiring passing lint checks or verifying that output files exist.

TeammateIdle hooks don't support matchers and fire on every occurrence.

TeammateIdle input

In addition to the **common input fields**, TeammateIdle hooks receive `teammate_name` and `team_name`.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9b...",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "TeammateIdle",
  "teammate_name": "researcher",
  "team_name": "session-a1b2c3d4"
}
```

Field	Description
<code>teammate_name</code>	Name of the teammate that is about to go idle
<code>team_name</code>	Deprecated. Session-derived team name; will be removed in a future release

instead of going idle.

- **JSON** `{"continue": false, "stopReason": "..."} : stops the teammate entirely, matching Stop hook behavior. The stopReason is shown to the user.`

This example checks that a build artifact exists before allowing a teammate to go idle:

```
#!/bin/bash

if [ ! -f "./dist/output.js" ]; then
    echo "Build artifact missing. Run the build before stopping." >&2
    exit 2
fi

exit 0
```

ConfigChange

Runs when a configuration file changes during a session. Use this to audit settings changes, enforce security policies, or block unauthorized modifications to configuration files.

Claude Code runs ConfigChange hooks when a settings file, a managed policy file, or a skill file changes. For managed policy, it runs them only when `managed-settings.json` or a file in `managed-settings.d/` changes. It applies **server-managed settings** and changes to macOS managed preferences or Windows registry policy without running them. On WSL with **wslInheritsWindowsSettings**, it also applies a changed Windows-side managed settings file on its policy poll without running them.

The matcher filters on the configuration source:

local_settings	.claude/settings.local.json changes
policy_settings	managed-settings.json or a file in managed-settings.d/ changes
skills	A skill file in .claude/skills/ changes

This example logs all configuration changes for security auditing:

```
{
  "hooks": {
    "ConfigChange": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/audit-config-change.sh",
            "args": []
          }
        ]
      }
    ]
  }
}
```

ConfigChange input

In addition to the **common input fields**, ConfigChange hooks receive `source` and optionally `file_path`. The `source` field indicates which configuration type changed, and `file_path` provides the path to the specific file that was modified.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9t",
  "cwd": "/Users/...",
  "hook_event_name": "ConfigChange",
  "source": "project_settings",
  "file_path": "/Users/.../my-project/.claude/settings.json"
}
```

Field	Description
<code>decision</code>	<code>"block"</code> prevents the configuration change from being applied. Omit to allow the change
<code>reason</code>	Accepted but never shown

```
{
  "decision": "block",
  "reason": "Configuration changes to project settings require admin approval"
}
```

`policy_settings` changes can't be blocked. Hooks still fire for `policy_settings` sources when a managed settings file on the machine changes, so you can use them to log those edits, but any blocking decision is ignored. This ensures enterprise-managed settings always take effect. Claude Code doesn't run `ConfigChange` hooks when server-managed settings arrive or refresh.

Claude Code acts on the blocking decision from a `ConfigChange` hook's JSON output and discards `systemMessage` and `continue`. A blocked change surfaces no message to you or to Claude, whether you block with `reason` or with `stderr` on exit 2. Claude Code only writes a line to the debug log.

CwdChanged

Runs when the working directory changes during a session, for example when Claude executes a `cd` command. Use this to react to directory changes: reload environment variables, activate project-specific toolchains, or run setup scripts automatically. Pairs with `FileChanged` for tools like `direnv` that manage per-directory environment.

`CwdChanged` hooks have access to `CLAUDE_ENV_FILE`. Variables written to that file persist into subsequent Bash commands for the session, just as in `SessionStart` hooks.

`CwdChanged` doesn't support matchers and fires on every directory change.

```
"session_id": "abc123",
"transcript_path": "/Users/.../.claude/projects/.../transcript.jsonl",
"cwd": "/Users/my-project/src",
"hook_event_name": "CwdChanged",
"old_cwd": "/Users/my-project",
"new_cwd": "/Users/my-project/src"
}
```

CwdChanged output

In addition to the **JSON output fields** available to all hooks, CwdChanged hooks can return `watchPaths` to dynamically set which file paths **FileChanged** watches:

Field	Description
<code>watchPaths</code>	Array of absolute paths. Replaces the current dynamic watch list. Paths from your <code>matcher</code> configuration are always watched. Returning an empty array clears the dynamic list, which is typical when entering a new directory

CwdChanged hooks have no decision control. They can't block the directory change.

Claude Code reads `watchPaths` and `systemMessage` from their JSON output and discards `continue`. In interactive sessions, it shows the `systemMessage` as a brief terminal notification. The message doesn't reach the SDK message stream.

DirectoryAdded

Runs after you add a working directory mid-session with the `/add-dir` command, or after an SDK client adds one with the `register_repo_root` control request. Use this to prepare a newly added repository, for example by installing its dependencies.

Claude Code doesn't fire this event when:

- You pass a directory with the `--add-dir` startup flag; **SessionStart** covers those directories
- You add a directory on the `/permissions` Workspace tab
- You add a directory that is already a working directory; the add fails with an error

Claude Code doesn't wait for the hook: the add completes immediately, and the hook runs in the background with the 600-second default timeout.

The matcher filters on how the directory was added:

Matcher	When it fires
<code>slash_command</code>	You add a directory with <code>/add-dir</code>
<code>register_repo_root</code>	An SDK client adds a directory with the <code>register_repo_root</code> control request

DirectoryAdded input

In addition to the common input fields, DirectoryAdded hooks receive `directory` and `source`.

Field	Description
<code>directory</code>	Absolute path of the directory that was added
<code>source</code>	How the directory was added, <code>"slash_command"</code> for <code>/add-dir</code> or <code>"register_repo_root"</code> for the SDK control request

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../transcript.jsonl",
  "cwd": "/Users/my-project",
  "hook_event_name": "DirectoryAdded",
  "directory": "/Users/my-other-repo",
  "source": "slash_command"
}
```

DirectoryAdded hooks have no decision control. They can't block the add, which has already completed when the hook runs. Claude Code discards the `continue` field from their JSON output and surfaces the rest differently per source:

- `slash_command`: Claude Code delivers the hook's `systemMessage` to Claude as context on the next conversation turn, rather than showing it to you. A count of failed hooks appears in the transcript. Full failure output goes to the debug log

Runs when a watched file changes on disk. Claude Code detects changes with a filesystem watcher, not by inspecting tool calls, so it runs the hook no matter what changed the file: an `Edit` or `Write` tool call, a script Claude runs with `Bash`, or a process outside Claude Code entirely. A common use is reloading environment variables when project configuration files change.

The `matcher` for this event serves two roles:

- **Build the watch list:** the value is split on `|` and each segment is registered as a literal filename in the working directory, so `".envrc|.env"` watches exactly those two files. Regex patterns are not useful here: a value like `^\.env` would watch a file literally named `^\.env`.
- **Filter which hooks run:** when a watched file changes, the same value filters which hook groups run using the standard matcher rules against the changed file's basename.

This example normalizes line endings in `data.csv` after any change, including a `Bash` command or an external script rewriting the file:

```
{
  "hooks": {
    "FileChanged": [
      {
        "matcher": "data.csv",
        "hooks": [
          {
            "type": "command",
            "command": "/path/to/normalize-line-endings.sh"
          }
        ]
      }
    ]
  }
}
```

The hook reads the changed file's absolute path from the `file_path` field of the JSON input on stdin. Its `grep` guard tests for the same thing `perl` removes, a CR at the end of a line, so the run after a normalization exits without touching the file. A looser guard loops forever, because `perl -i` rewrites the file even when it substitutes nothing and Claude Code runs the hook again after every rewrite. Save this script at `/path/to/normalize-line-endings.sh` and make it executable:

T1

To confirm the hook works, ask Claude to append a CRLF line to `data.csv` with a `Bash` command. Claude Code runs the hook and the file ends up with LF endings.

To watch files you can't name up front, return `watchPaths` from a hook to update the watch list dynamically. Claude Code starts the watcher only when something names a file to watch, so seed the list with a `FileChanged` group whose matcher names at least one file, or with a `SessionStart` or `CwdChanged` hook that returns `watchPaths`. The matcher still filters which hook groups run when a watched file changes, so give the group that handles dynamic paths an omitted matcher, which matches every watched file and adds nothing to the watch list. A `"*"` matcher also matches every file, but Claude Code registers it in the watch list like any other value, as a literal file named `*`.

`FileChanged` hooks have access to `CLAUDE_ENV_FILE`. Variables written to that file persist into subsequent `Bash` commands for the session, just as in [SessionStart hooks](#).

FileChanged input

In addition to the [common input fields](#), `FileChanged` hooks receive `file_path` and `event`.

Field	Description
<code>file_path</code>	Absolute path to the file that changed
<code>event</code>	What happened: <code>"change"</code> for a modified file, <code>"add"</code> for a created file, or <code>"unlink"</code> for a deleted file

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../transcript.jsonl",
  "cwd": "/Users/my-project",
  "hook_event_name": "FileChanged",
  "file_path": "/Users/my-project/.envrc",
  "event": "change"
}
```

Field	Description
<code>watchPaths</code>	Array of absolute paths. Replaces the current dynamic watch list. Paths from your <code>matcher</code> configuration are always watched. Use this when your hook script discovers additional files to watch based on the changed file

FileChanged hooks have no decision control. They can't block the file change from occurring.

Claude Code reads `watchPaths` and `systemMessage` from their JSON output and discards `continue`. In interactive sessions, it shows the `systemMessage` as a brief terminal notification. The message doesn't reach the SDK message stream.

WorktreeCreate

Runs when a worktree is being created, whether from `claude --worktree`, from a subagent using `isolation: "worktree"`, or for a background session that Claude Code isolates in its own worktree. By default Claude Code creates the isolated working copy with `git worktree`. Configuring a WorktreeCreate hook replaces that default git behavior, letting you use a different version control system like SVN, Perforce, or Mercurial.

Because the hook replaces the default behavior entirely, `.worktreeinclude` is not processed. If you need to copy local configuration files like `.env` into the new worktree, do it inside your hook script.

The hook must return the path to the created worktree directory. Claude Code uses this path as the working directory for the isolated session. See WorktreeCreate output for how each hook type returns the path.

Claude Code acts on the hook's success and the returned path, and discards `systemMessage` and `continue`.

This example creates an SVN working copy and prints the path for Claude Code to use. Replace the repository URL with your own:

```
    "hooks": [
      {
        "type": "command",
        "command": "bash -c 'NAME=$(jq -r .name); DIR=\"${HOME}/.claude/worktrees/${NAME}\"; echo $DIR';"
      }
    ]
  }
}
```

The hook reads the worktree `name` from the JSON input on stdin, checks out a fresh copy into a new directory, and prints the directory path. The `echo` on the last line is what Claude Code reads as the worktree path. Redirect any other output to stderr so it doesn't interfere with the path.

WorktreeCreate input

In addition to the common input fields, WorktreeCreate hooks receive the `name` field. This is a slug identifier for the new worktree, either specified by the user or auto-generated, for example `bold-oak-a3f2`.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9b...",
  "cwd": "/Users/...",
  "hook_event_name": "WorktreeCreate",
  "name": "feature-auth"
}
```

WorktreeCreate output

WorktreeCreate hooks don't use the standard allow/block decision model. Instead, the hook's success or failure determines the outcome. The hook must return the path to the created worktree directory:

- **Command hooks** (`type: "command"`): print the path as the last non-empty line of stdout. Claude Code strips ANSI escape codes before reading that line, so shell startup banners printed before your `echo` are ignored. Redirect any other hook output to stderr.

Claude Code resolves a relative path against the directory the hook ran in, collapsing any `.` or `..` segments in it. If the resulting path isn't a directory Claude Code can enter, the session prints an error naming the path and exits with code 1.

Claude Code refuses an absolute path that contains `.` or `..` segments, and any path that passes through a symlink below the repository root, because a symlink committed to the repository could redirect the worktree outside it. The error names the rejected component. Return a normalized path that doesn't pass through a symlink inside the repository. Before v2.1.216, worktree creation followed the hook's path without this screening.

WorktreeRemove

Runs when a worktree is being removed. This is the cleanup counterpart to [WorktreeCreate](#). The event fires when:

- you exit a `--worktree` session and choose to remove it
- a subagent with `isolation: "worktree"` finishes
- you delete a [background session](#) whose worktree the hook created

For git-based worktrees, Claude Code handles cleanup automatically with `git worktree remove`. If you configured a WorktreeCreate hook for a non-git version control system, pair it with a WorktreeRemove hook to handle cleanup. Without one, the worktree directory is left on disk.

Claude Code discards a WorktreeRemove hook's [JSON output fields](#), such as `systemMessage` and `continue`.

For a background-session delete, Claude Code verifies the stored worktree path before running the hook and refuses a path that is a symlink or passes through one below the repository root. The hook runs for a worktree that still contains files only when you confirm the delete in [agent view](#); for such a worktree, `claude rm` keeps the session and worktree instead. Before v2.1.216, the hook ran on the stored path without these checks.

Claude Code passes the path returned by WorktreeCreate as `worktree_path` in the hook input. This example reads that path and removes the directory:

```
    "hooks": [
      {
        "type": "command",
        "command": "bash -c 'jq -r .worktree_path | xargs rm -rf'"
      }
    ]
  }
]
```

WorktreeRemove input

In addition to the **common input fields**, WorktreeRemove hooks receive the `worktree_path` field, which is the absolute path to the worktree being removed.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9b...",
  "cwd": "/Users/...",
  "hook_event_name": "WorktreeRemove",
  "worktree_path": "/Users/.../my-project/.claude/worktrees/feature-auth"
}
```

WorktreeRemove hooks have no decision control. They can't block worktree removal but can perform cleanup tasks like removing version control state or archiving changes. Hook failures are logged in debug mode only.

PreCompact

Runs before Claude Code is about to run a compact operation.

The matcher value indicates whether compaction was triggered manually or automatically:

Exit with code 2 to block compaction. For a manual `/compact`, the stderr message is shown to the user. You can also block by returning JSON with `"decision": "block"`.

Blocking automatic compaction has different effects depending on when it fires. If compaction was triggered proactively before the context limit, Claude Code skips it and the conversation continues uncompacted. If compaction was triggered to recover from a context-limit error already returned by the API, the underlying error surfaces and the current request fails.

Claude Code discards a PreCompact hook's `systemMessage` and `continue` fields.

PreCompact input

In addition to the common input fields, PreCompact hooks receive `trigger` and `custom_instructions`. For `manual`, `custom_instructions` contains what the user passes into `/compact` and is `null` when they pass nothing. For `auto`, `custom_instructions` is `null`.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9k",
  "cwd": "/Users/...",
  "hook_event_name": "PreCompact",
  "trigger": "manual",
  "custom_instructions": null
}
```

PostCompact

Runs after Claude Code completes a compact operation. Use this event to react to the new compacted state, for example to log the generated summary or update external state. Claude Code discards a PostCompact hook's `systemMessage` and `continue` fields.

The same matcher values apply as for `PreCompact`:

Matcher	When it fires
<code>manual</code>	After <code>/compact</code>

PostCompact input

In addition to the **common input fields**, PostCompact hooks receive `trigger` and `compact_summary`. The `compact_summary` field contains the conversation summary generated by the compact operation.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9b",
  "cwd": "/Users/...",
  "hook_event_name": "PostCompact",
  "trigger": "manual",
  "compact_summary": "Summary of the compacted conversation..."
}
```

PostCompact hooks have no decision control. They can't affect the compaction result but can perform follow-up tasks.

SessionEnd

Runs when a Claude Code session ends. Useful for cleanup tasks, logging session statistics, or saving session state. Supports matchers to filter by exit reason.

The `reason` field in the hook input indicates why the session ended:

Reason	Description
<code>clear</code>	Session cleared with <code>/clear</code> command
<code>resume</code>	Session switched via interactive <code>/resume</code>
<code>logout</code>	User logged out
<code>prompt_input_exit</code>	User exited while prompt input was visible
<code>other</code>	Other exit reasons
<code>bypass_permissions_disabled</code>	Removed in v2.1.234; Claude Code doesn't send it. Drop it from your <code>SessionEnd</code> matchers

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9b",
  "cwd": "/Users/...",
  "hook_event_name": "SessionEnd",
  "reason": "other"
}
```

SessionEnd hooks have no decision control. They can't block session termination but can perform cleanup tasks. Claude Code discards their **JSON output fields**, such as `systemMessage`.

SessionEnd hooks have a default timeout of 1.5 seconds. This applies to session exit, `/clear`, and switching sessions via interactive `/resume`. If a hook needs more time, set a per-hook `timeout` in the hook configuration. The overall budget is automatically raised to the highest per-hook timeout configured in settings files, up to 60 seconds. Timeouts set on plugin-provided hooks don't raise the budget. To override the budget explicitly, set the `CLAUDE_CODE_SESSIONEND_HOOKS_TIMEOUT_MS` environment variable in milliseconds.

```
CLAUDE_CODE_SESSIONEND_HOOKS_TIMEOUT_MS=5000 claude
```

Elicitation

Runs when an MCP server requests user input mid-task. By default, Claude Code shows an interactive dialog for the user to respond. Hooks can intercept this request and respond programmatically, skipping the dialog entirely.

The `matcher` field matches against the MCP server name.

Elicitation input

In addition to the **common input fields**, Elicitation hooks receive `mcp_server_name`, `message`, and optional `mode`, `url`, `elicitation_id`, and `requested_schema` fields.

For form-mode elicitation, the most common case:

```
"permission_mode": "default",
"hook_event_name": "Elicitation",
"mcp_server_name": "my-mcp-server",
"message": "Please provide your credentials",
"mode": "form",
"requested_schema": {
  "type": "object",
  "properties": {
    "username": { "type": "string", "title": "Username" }
  }
}
}
```

For URL-mode elicitation, used for browser-based authentication:

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-19fa-41d2-8238-13269b9b",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "Elicitation",
  "mcp_server_name": "my-mcp-server",
  "message": "Please authenticate",
  "mode": "url",
  "url": "https://auth.example.com/login"
}
```

Elicitation output

To respond programmatically without showing the dialog, return a JSON object with

`hookSpecificOutput` :

```
"content": {  
  "username": "alice"  
}  
}  
}
```

Field	Values	Description
<code>action</code>	<code>accept</code> , <code>decline</code> , <code>cancel</code>	Whether to accept, decline, or cancel the request
<code>content</code>	object	Form field values to submit. Only used when <code>action</code> is <code>accept</code>

Exit code 2 denies the elicitation. Claude Code doesn't show your stderr message anywhere.

Claude Code acts on `hookSpecificOutput` from an Elicitation hook's JSON output and discards `systemMessage` and `continue` .

ElicitationResult

Runs after a user responds to an MCP elicitation. Hooks can observe, modify, or block the response before it is sent back to the MCP server.

The matcher field matches against the MCP server name.

ElicitationResult input

In addition to the **common input fields**, ElicitationResult hooks receive `mcp_server_name` , `action` , and optional `mode` , `elicitation_id` , and `content` fields.

```
"permission_mode": "default",
"hook_event_name": "ElicitationResult",
"mcp_server_name": "my-mcp-server",
"action": "accept",
"content": { "username": "alice" },
"mode": "form",
"elicitation_id": "elicit-123"
}
```

ElicitationResult output

To override the user's response, return a JSON object with `hookSpecificOutput` :

```
{
  "hookSpecificOutput": {
    "hookEventName": "ElicitationResult",
    "action": "decline",
    "content": {}
  }
}
```

Field	Values	Description
<code>action</code>	<code>accept</code> , <code>decline</code> , <code>cancel</code>	Overrides the user's action
<code>content</code>	object	Overrides form field values. Only meaningful when <code>action</code> is <code>accept</code>

Exit code 2 blocks the response, changing the effective action to `decline` . Claude Code doesn't show your stderr message anywhere.

Claude Code acts on `hookSpecificOutput` from an ElicitationResult hook's JSON output and discards `systemMessage` and `continue` .

hooks (type: `agent`) that spawn an agentic vendor with tool access. Not all events support every hook type.

Events that support all five hook types (`command` , `http` , `mcp_tool` , `prompt` , and `agent`):

- `PermissionDenied`
- `PermissionRequest`
- `PostToolBatch`
- `PostToolUse`
- `PostToolUseFailure`
- `PreToolUse`
- `Stop`
- `SubagentStop`
- `TaskCompleted`
- `TaskCreated`
- `TeammateIdle`
- `UserPromptExpansion`
- `UserPromptSubmit`

Events that support `command` , `http` , and `mcp_tool` hooks but not `prompt` or `agent` :

- `ConfigChange`
- `CwdChanged`
- `DirectoryAdded`
- `Elicitation`
- `ElicitationResult`
- `FileChanged`
- `InstructionsLoaded`
- `MessageDisplay`
- `Notification`
- `PostCompact`

- `SubagentStart`
- `WorktreeCreate`
- `WorktreeRemove`

`SessionStart` and `Setup` support `command` and `mcp_tool` hooks. They don't support `http`, `prompt`, or `agent` hooks.

How prompt-based hooks work

Instead of executing a Bash command, prompt-based hooks:

1. Send the hook input and your prompt to a Claude model, Haiku by default
2. The LLM responds with structured JSON containing a decision
3. Claude Code processes the decision automatically

Prompt hook configuration

Set `type` to `"prompt"` and provide a `prompt` string instead of a `command`. Use the `$ARGUMENTS` placeholder to inject the hook's JSON input data into your prompt text.

This `Stop` hook asks the LLM to evaluate whether all tasks are complete before allowing Claude to finish:

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "prompt",
            "prompt": "Evaluate if Claude should stop: $ARGUMENTS. Check if all tasks"
          }
        ]
      }
    ]
  }
}
```

		placeholder for the hook input JSON. If <code>\$ARGUMENTS</code> is not present, input JSON is appended to the prompt
model	no	Model to use for evaluation. Defaults to a fast model
timeout	no	Timeout in seconds. Default: 30
continueOnBlock	no	On the events it applies to, <code>true</code> feeds an <code>ok: false</code> reason back to Claude and continues instead of ending the turn. Default: <code>false</code> . See Response schema for per-event behavior

Response schema

The LLM must respond with JSON containing:

```
{
  "ok": true | false,
  "reason": "Explanation for the decision",
  "impossible": true | false
}
```

Field	Description
ok	<code>true</code> to allow. For <code>false</code> , see the per-event behavior below
reason	Required when <code>ok</code> is <code>false</code>
impossible	Optional. The model returns it with <code>ok: false</code> when it judges the condition can never be satisfied. On <code>Stop</code> and <code>SubagentStop</code> , Claude Code then lets the turn end instead of feeding the reason back. Agent hooks and other events ignore it

What happens on `ok: false` depends on the event:

- `Stop` and `SubagentStop`: the reason is fed back to Claude as its next instruction and the turn continues, unless the response also sets `impossible: true`, in which case Claude Code allows the stop and the turn ends

tool error and the turn continued

- `PostToolUse` : by default the turn ends and the reason appears in the chat as a warning line. Set `continueOnBlock: true` to feed the reason back to Claude and continue the turn instead
- `PostToolBatch` , `UserPromptSubmit` , and `UserPromptExpansion` : the turn ends and the reason appears as a warning line. These events end the turn on `decision: "block"` regardless of `continue`
- `PostToolUseFailure` and `TaskCreated` : the reason is returned to Claude as a tool error and the turn continues, regardless of `continueOnBlock`
- `TaskCompleted` : when it fires because a task is marked completed during a turn, the reason is returned to Claude as a tool error and the turn continues, regardless of `continueOnBlock` . When it fires because a teammate stops, it behaves like `TeammateIdle` and halts the teammate by default
- `TeammateIdle` : by default the teammate stops and the reason appears as a warning line. Set `continueOnBlock: true` to feed the reason back to the teammate and keep it working instead
- `PermissionRequest` : `ok: false` has no effect. To deny an approval from a hook, use a **command hook** returning `hookSpecificOutput.decision.behavior: "deny"`
- `PermissionDenied` : `ok: false` has no effect because the denial already happened. The only output this event reads is `hookSpecificOutput.retry` , which prompt and agent hooks can't set. They run on this event, but their output is discarded. Use a **command hook** to return `retry`

If you need finer control on any event, use a **command hook** with the per-event fields described in **Decision control**.

Check multiple conditions before stopping

This `Stop` hook uses a detailed prompt to check three conditions before allowing Claude to stop.

`SubagentStop` hooks use the same format to evaluate whether a **subagent** should stop. If the model returns `"ok": false` because the condition isn't met yet, Claude continues working with the provided reason as its next instruction:

```
    "hooks": [
      {
        "type": "prompt",
        "prompt": "You are evaluating whether Claude should stop working. Context:",
        "timeout": 30
      }
    ]
  }
]
```

Agent-based hooks

⚠ Agent hooks are experimental. Behavior and configuration may change in future releases. For production workflows, prefer [command hooks](#).

Agent-based hooks (`type: "agent"`) are like prompt-based hooks but with multi-turn tool access. Instead of a single LLM call, an agent hook spawns a subagent that can read files, search code, and inspect the codebase to verify conditions. Agent hooks support the same events as prompt-based hooks.

How agent hooks work

When an agent hook fires:

1. Claude Code spawns a subagent with your prompt and the hook's JSON input
2. The subagent can use tools like Read, Grep, and Glob to investigate
3. After up to 50 turns, the subagent returns a structured `{ "ok": true/false }` decision
4. Claude Code allows the action if `ok is true` . If `ok is false` , Claude Code handles the block the same way as a prompt hook with `continueOnBlock: true` on that event, as listed under [Response schema](#)

Agent hooks are useful when verification requires inspecting actual files or test output, not just evaluating the hook input data alone.

Field	Required	Description
<code>type</code>	yes	Must be <code>"agent"</code>
<code>prompt</code>	yes	Prompt describing what to verify. Use <code>\$ARGUMENTS</code> as a placeholder for the hook input JSON
<code>model</code>	no	Model to use. Defaults to a fast model
<code>timeout</code>	no	Timeout in seconds. Default: 60

The response schema is `{ "ok": true }` to allow or `{ "ok": false, "reason": "..." }` to block. On `ok: false`, Claude Code handles an agent hook the way it handles a prompt hook with `continueOnBlock: true` on the same event; agent hooks have no `continueOnBlock` field, and don't support the prompt-hook `impossible` field.

This `Stop` hook verifies that all unit tests pass before allowing Claude to finish:

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "agent",
            "prompt": "Verify that all unit tests pass. Run the test suite and check t",
            "timeout": 120
          }
        ]
      }
    ]
  }
}
```

Run hooks in the background

By default, hooks block Claude's execution until they complete. For long-running tasks like deployments, test suites, or external API calls, set `"async": true` to run the hook in the

Configure an async hook

Add `"async": true` to a command hook's configuration to run it in the background without blocking Claude. This field is only available on `type: "command"` hooks.

This hook runs a test script after every `Write` tool call. Claude continues working immediately while `run-tests.sh` executes. When the script finishes, its output is delivered on the next conversation turn:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write",
        "hooks": [
          {
            "type": "command",
            "command": "/path/to/run-tests.sh",
            "async": true
          }
        ]
      }
    ]
  }
}
```

Once an async hook is running in the background, Claude Code doesn't enforce `timeout` on it. Claude Code still enforces `timeout` on a hook you run with `asyncRewake`.

Claude Code delivers an async hook's results only while the session runs:

- In **non-interactive mode** with the `-p` flag, Claude Code kills any async hook still running at teardown and finalizes it with outcome `cancelled`
- If your hook's work must outlive a `claude -p` session, start a fully detached process from it

After the background process exits, Claude Code delivers the `additionalContext` and `systemMessage` fields from the hook's JSON response to Claude on the next conversation turn. Unlike a synchronous hook's `systemMessage`, neither field is shown to you.

Claude Code validates that JSON response against the same **output schema** as synchronous hooks, and drops any field whose value has the wrong type, such as a `systemMessage` that isn't a string, instead of delivering it. Run with `--debug` to see a warning naming each dropped field. Before v2.1.202, malformed JSON output from an async hook could crash the session, and the crash recurred each time the session was resumed.

Async hook completion notifications are suppressed by default. To see them, enable verbose mode with `Ctrl+0` or start Claude Code with `--verbose`.

Run tests after file changes

This hook starts a test suite in the background whenever Claude writes a file, then reports the results back to Claude when the tests finish. Save this script to `.claude/hooks/run-tests-async.sh` in your project and make it executable with `chmod +x`:

```
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

# Only run tests for source files
if [[ "$FILE_PATH" != *.ts && "$FILE_PATH" != *.js ]]; then
    exit 0
fi

# Run tests and report results to Claude via additionalContext
RESULT=$(npm test 2>&1)
EXIT_CODE=$?

if [ $EXIT_CODE -eq 0 ]; then
    MSG="Tests passed after editing $FILE_PATH"
else
    MSG="Tests failed after editing $FILE_PATH: $RESULT"
fi
jq -nc --arg msg "$MSG" '{hookSpecificOutput: {hookEventName: "PostToolUse", additionalContext: $msg}}
```

Then add this configuration to `.claude/settings.json` in your project root. The `async: true` flag lets Claude keep working while tests run:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/run-tests-async.sh",
            "args": [],
            "async": true
          }
        ]
      }
    ]
  }
}
```

until the next user interaction. Exception: an `asyncRewake` hook that exits with code 2 wakes Claude immediately even when the session is idle.

- Each execution creates a separate background process. There is no deduplication across multiple firings of the same async hook.

Security considerations

Disclaimer

⚠ Command hooks execute shell commands with your full user permissions. They can modify, delete, or access any files your user account can access. Review and test all hook commands before adding them to your configuration.

Workspace trust

Claude Code checks workspace trust before it runs any hook from a settings file. What counts as trusted depends on the session type:

- **Interactive session:** Claude Code holds back hooks from every settings file, including your own `~/.claude/settings.json`, until you accept the workspace trust dialog for the folder, or for a parent directory whose trust extends to it
- **-p or SDK session:** Claude Code never shows the dialog and treats the folder as trusted, so hooks committed in a repository's `.claude/settings.json` run in a folder you've never trusted

Before you script `claude -p` over a repository you didn't write, review its `.claude/` settings files, start with `--bare`, or turn hooks off for that run with `--settings '{"disableAllHooks": true}'`. Frontmatter hooks in a project subagent follow a stricter rule than settings-file hooks.

What runs before you trust a folder lists each kind of repository content by session type.

Security best practices

Keep these practices in mind when writing hooks:

- **Validate and sanitize inputs:** never trust input data blindly

- **Skip sensitive files:** avoid `.env` , `.git/` , keys, etc.

Windows PowerShell tool

On Windows, you can run individual hooks in PowerShell by setting `"shell": "powershell"` on a command hook. Claude Code auto-detects `pwsh.exe` , the PowerShell 7 and later executable, and falls back to `powershell.exe` for Windows PowerShell 5.1.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write",
        "hooks": [
          {
            "type": "command",
            "shell": "powershell",
            "command": "Write-Host 'File written'"
          }
        ]
      }
    ]
  }
}
```

To reference the project root from a PowerShell shell-form command, write

`${CLAUDE_PROJECT_DIR}` or `$env:CLAUDE_PROJECT_DIR` . As of v2.1.198, Claude Code rewrites the `${CLAUDE_PROJECT_DIR}` , `${CLAUDE_PLUGIN_ROOT}` , and `${CLAUDE_PLUGIN_DATA}` placeholders in a PowerShell shell-form command to PowerShell's `${env:NAME}` form, whether the hook is defined in `settings.json` , a plugin, or a skill. PowerShell then resolves the value from the exported environment after parsing, so the placeholder works inside double-quoted strings but not inside single-quoted strings, where PowerShell never expands variables.

Before v2.1.198, this rewrite applied only to plugin hooks. On earlier versions, a `settings.json` hook needs the `$env:` form or **exec form**, where `${CLAUDE_PROJECT_DIR}` is substituted in each `args` element regardless of where the hook is defined.

The example below shows a `settings.json` hook that runs a project script with the `$env:` form, which works on every version:

```
{
  "type": "command",
  "shell": "powershell",
  "command": "& \"$env:CLAUDE_PROJECT_DIR\\.claude\\hooks\\check.ps1\""
}
```

Debug hooks

Hook execution details, including which hooks matched, their exit codes, and full stdout and stderr, are written to the debug log file. Start Claude Code with `claude --debug-file <path>` to write the log to a known location, or run `claude --debug` and read the log at

`~/.claude/debug/<session-id>.txt`. The `--debug` flag doesn't print to the terminal.

For example, a `PostToolUse` hook on `Write` whose command prints `hook-ran` produces entries like:

```
2026-07-19T02:03:24.382Z [DEBUG] Hook output does not start with {, treating as plain
2026-07-19T02:03:24.382Z [DEBUG] Hook PostToolUse:Write (PostToolUse) success:
hook-ran
```

For more granular hook matching details, set `CLAUDE_CODE_DEBUG_LOG_LEVEL=verbose` to see additional log lines such as hook matcher counts and query matching.

For troubleshooting common issues like hooks not firing, Stop hooks that keep blocking, or configuration errors, see [Limitations and troubleshooting](#) in the guide. For a broader diagnostic walkthrough covering `/context`, `/doctor`, and settings precedence, see [Debug your config](#).

Was this page helpful?

 Yes

 No

Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

[News](#)

[Trust center](#)

[Transparency](#)

Learn

[Courses](#)

[MCP connectors](#)

[Customer stories](#)

[Engineering blog](#)

[Events](#)

[Powered by Claude](#)

[Service partners](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

Terms and policies

[Privacy choices](#)

[Privacy policy](#)

[Disclosure policy](#)

[Usage policy](#)

[Commercial terms](#)

[Consumer terms](#)

